Manual

Creating/ Managing Form

FEP-EVF 8.1

Version 1.1.1374

Last changed on: 19.06.2020

Copyright

Contents

1 Creating/ Managing Form

Purpose

This component is used when you want to be able to create your own reports in Word. This component is often used to create inspection certificates based on input inspection data.

Implementation considerations

It is a good idea to have a license for this component if the installed standard reports do not meet your own or you customers' requirements.

Integration

This component can relate to a range of other different components. Its primary purpose is to combine the "Inspection planning for in-production inspections" component with the ability to prepare inspection plan forms and inspection certificates.

Features

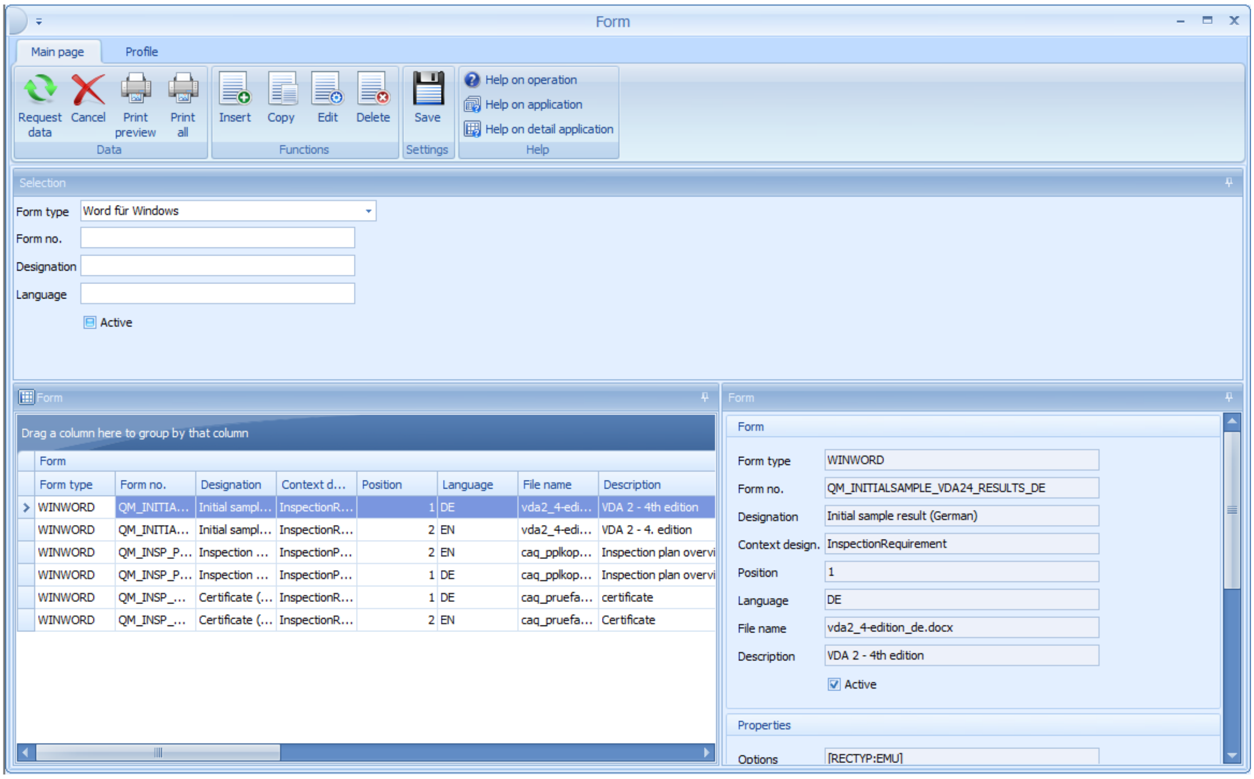
The following functions are available:

- Manage Word reports with the option to deactivate and/or release, modify form designations and positions in the print list and to define the print location (screen, e-mail, printer).
- Modify existing standard reports to create new reports based on customer specifications.
- Create completely new forms in Word
- Use the draft versions to modify existing forms
- Use Word functions to optimize the design, e.g. to integrate your own Macros.

2 Forms

Summary

Menu	Master data → Quality management → Form
Transaction code	form
Function authorization	Form If you have the authorization “form.design” the print selection dialog also shows entries/reports that have not yet been released. This enables you to design and test reports before you publish them.



The catalog of forms has been designed to manage CAQ reports. New form entries may be created and existing entries can be changed with respect to their options or descriptions. A new form entry is the basis for the creation/designing of a new report. The report design is not part of this application.

Utilization

Some CAQ applications, e.g. inspection planning, inspection requirement and failure analysis of complaint management allow for a context-related list of Word forms to be opened using a special “print button” in the toolbar. As these Word forms are not just based on the available list data, an export program is required that “collects” appropriate data and makes them available to the form. Each Word form needs such an export program. Vice versa, many different forms can be created on the basis of an export program.

Export programs have been designed in a way so as to provide an as large amount of data as possible. For example, the export program that is responsible for printing inspection plans including characteristics exports the essential header data of the inspection plan and the lower-level characteristics. Consequently, different inspection plan forms can be created on this basis for any purpose. The same applies to forms for printing inspection results/certificates.

The context of the corresponding application determines which forms are suggested for printing in the relevant application. The context “InspectionPlan” applies for inspection planning, the context “InspectionRequirment” applies for inspection requirements and “ComplaingManagement” applies for the complaint module. One or several export programs are available subject to the context.

Integration

The below applications use the contents of this form catalog

- Inspection planning (goods receipt, production, goods issue, gages, initial sample)
- Inspection requirement (goods receipt, production, goods issue, gages, initial sample) and
- Complaint management (failure analysis)

Prerequisites

The Word versions Microsoft Office 2010 or 2012 are required for using this function of creating/designing new forms or changing the design of existing forms. Word is not needed if the entries of this catalog are only edited/maintained.

Selection criteria

The selection criteria are not described separately as they are self-explanatory.

Field descriptions

Form type

Form type; HYDRA specifies the selection, only the type “Word for Windows” is supported at the moment.

Designation

Designation of the form as displayed in the print dialog

Context des.

Context in which the form is to be printed.

Form no.

Unique form ID

File name

The file name of the HYDRA Word Report to be designed needs to be entered at first in the “file name” field. Then all macro libraries in use have to be entered, separated by semicolon. Normally, only the HYDRA macro library is used (`HydraMacroLibrary.dotm`). Moreover, the form number must not start with the terms “TABELLE_“, „CAQ_“ or „GANT_“, as these are reserved prefixes.

Language

Language ID (e.g. DE, EN) for the user’s information. The language specified here is not related to the language configured for the console. Foreign-language forms may have a language ID that does neither depend on the form ID nor the designation.

Position

Numeric specification of which position the form should have in the list of printable forms. The list of the print dialog does not automatically sort by the position number. It is up to the user’s decision by which column the content is to be sorted. It is not checked if the position number is unique.

Print destination “e-mail“, “file“, “screen“ and “printer“

The activation of the print destination defines which destinations will later be available for this form for printing in the corresponding application. If only the “screen” option is enabled it can be achieved, for example, that the report needs to be opened on the screen for reviewing the content. Only then can printing be triggered manually.

Additional parameter

Additional parameters provide further control options for printing forms. Forms provided by HYDRA CAQ can include control parameters. The export program provides information on possible control parameters if new forms are created or existing forms are changed by the user. Further details on the relevant export programs are described in separate documents.

Export program

Export program providing the data basis and that is assigned to the form

Export group

Export group assigned to the form

Active

Identifying the form as active/inactive. Only active forms may be selected for printing

Description

Detailed description of the form (max. 250 characters); the description is displayed at the bottom of the print dialog when selecting the form

Options (number of copies / from page / to page)

The activation of options defines which options will later be available for this form when it is printed in the corresponding application.

Toolbar

Besides the standard functions, there are no other special function buttons.

3 Creating Word Reports

3.1 Utilization

You would like to change existing HYDRA Word Reports or create new HYDRA Word Reports.

All screenshots and descriptions included in this document refer to Microsoft Word 2010. If other versions of Word are used the dialogs and methods may slightly differ.

3.2 Prerequisites

You use Microsoft Word in version 2007 or 2010.

The settings in the Word Trust Center (can be found in the Word menu File --> Options --> Trust Center --> Trust Center Settings) are configured so as to enable in general the execution of macros for files in the MOC reporting directory ([MOC user data directory]\`export`) (with confirmation prompt, if necessary).

There is an entry for your HYDRA Word Report in Master data --> Forms

Please note: The file name of the HYDRA Word Report to be designed has to be entered first in the "file name" field.
All macro libraries that are in use have to be entered afterwards, separated by semicolon. Normally, only the HYDRA macro library (`HydraMacroLibrary.dotm`) is used. Moreover, the form number must not start with the terms "TABELLE_“, "CAQ_“ or "GANT_“, as these are reserved prefixes.

You have selected this HYDRA Word Report from the corresponding context for the report that is currently to be designed. Consequently, corresponding sample data has been generated which may be used for designing your Word Report.

Please note: By authorizing `form.design` you can also view entries of external reports that are not released in the selection dialog for printing.
This allows you to design and test the report before it is made available to other users.

3.3 Basics

3.3.1 Valuable information on starting a HYDRA Word Report from MOC

To be able to design a HYDRA Word Report, you need to understand the way they are started from MOC. This basic knowledge is described in the sections that follow.

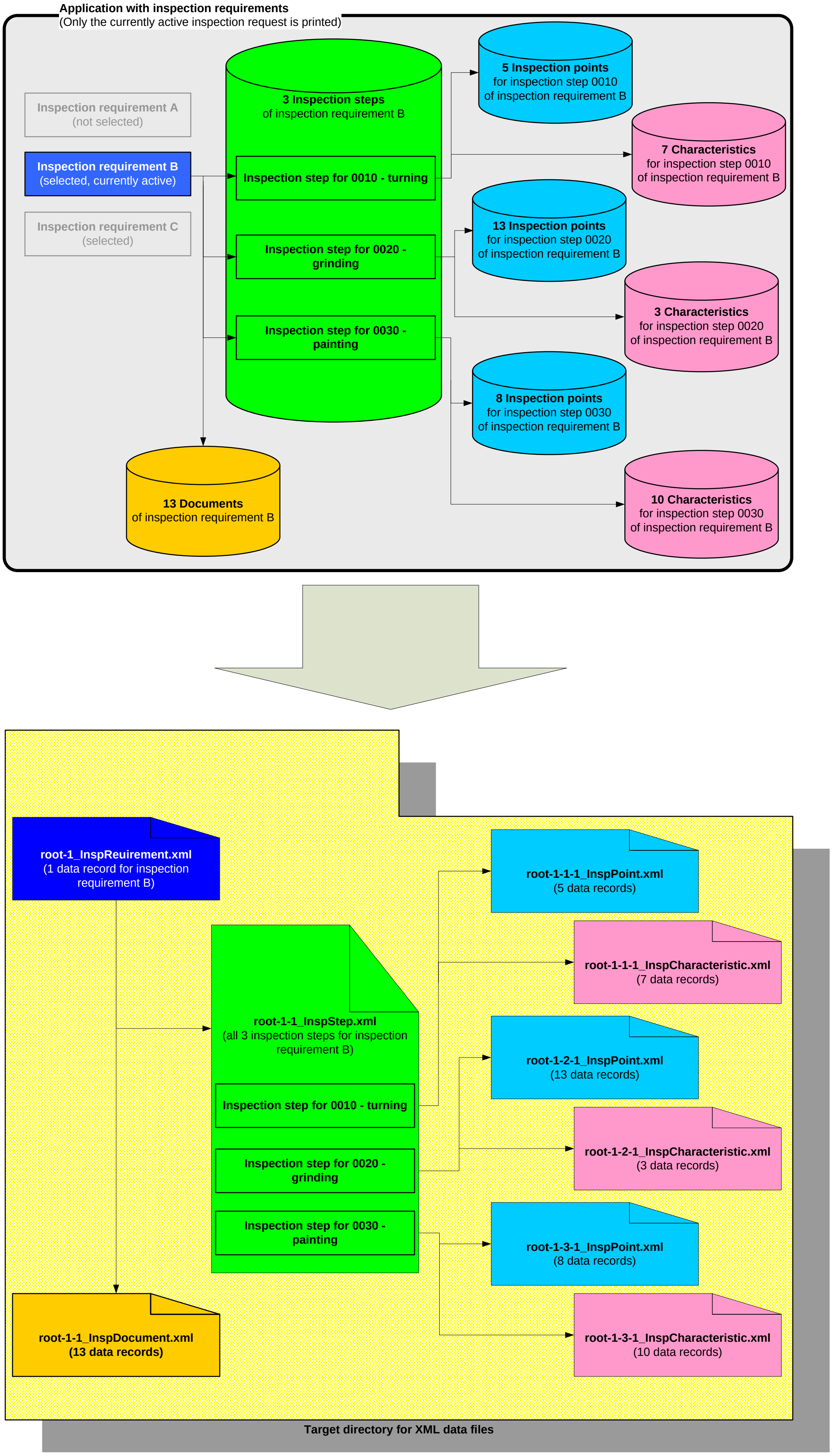
3.3.1.1 Loading of all files required for the HYDRA Word Report

This section describes all processes in detail that take place in MOC, once the HYDRA Word Report to be output has been selected and confirmed.

At first all files needed for the HYDRA Word Report are downloaded from the server to the client. In the majority of cases, this is only the HYDRA Word Report and the HYDRA Macro Library (`HydraMacroLibrary.dotm`) that provides all default functions for designing and outputting reports.
The corresponding detailed names are provided in the "file name" field of the form management function.
The subdirectory `.\`***[HYDRA System]***`\custom\caq\reports` of the HYDRA server is used at first as source directory for each file to be loaded. In case the file cannot be found here, it is searched in the subdirectory `.\db_ace`. A corresponding error message is displayed, provided that the file cannot be found there either.
The target directory for the files to be loaded is the sub-folder `.\Export` of the MOC user data directory.

3.3.1.2 Deletion of old export files and print control information

All XML and XSD files are deleted from the previously-mentioned directory to make sure that the HYDRA Word Report does not use data of previous print calls.


Application with inspection requirements
(Only the currently active inspection request is printed)
Inspection requirement A
(not selected)
Inspection requirement B
(selected, currently active)
Inspection requirement C
(selected)
3 Inspection steps
of inspection requirement B
Inspection step for 0010 - turning
Inspection step for 0020 - grinding
Inspection step for 0030 - painting
5 Inspection points
for inspection step 0010
of inspection requirement B
7 Characteristics
for inspection step 0010
of inspection requirement B
13 Inspection points
for inspection step 0020
of inspection requirement B
3 Characteristics
for inspection step 0020
of inspection requirement B
8 Inspection points
for inspection step 0030
of inspection requirement B
10 Characteristics
for inspection step 0030
of inspection requirement B
13 Documents
of inspection requirement B
root-1_InspReuirement.xml
(1 data record for inspection requirement B)
root-1-1_InspStep.xml
(all 3 inspection steps for inspection requirement B)
Inspection step for 0010 - turning
Inspection step for 0020 - grinding
Inspection step for 0030 - painting
root-1-1-1_InspPoint.xml
(5 data records)
root-1-1-1_InspCharacteristic.xml
(7 data records)
root-1-2-1_InspPoint.xml
(13 data records)
root-1-2-1_InspCharacteristic.xml
(3 data records)
root-1-3-1_InspPoint.xml
(8 data records)
root-1-3-1_InspCharacteristic.xml
(10 data records)
root-1-1_InspDocument.xml
(13 data records)
Target directory for XML data files

3.3.1.3 Output of print control information

Then all pieces of information available for the selected report as well as the details specified by the user for outputting the report (print target, number of copies etc.) are written in a separate XML file.

This XML file is stored with the same file name and within the same directory as it is the case for the HYDRA Word Report. Only the file extension (,`xml`) distinguishes this file from the Word Report (.dotm).

3.3.1.4 Output of HYDRA information to be printed

All detail information relevant for the HYDRA Word Report are determined and output in XML files. The export application which is also stored within form management determines which data is output and in which form.

The structure of the data to be exported and the XML files resulting from it is described in the sections that follow on the basis of a sample application. However, the actual structure of exported data depends on the corresponding MOC export application.

3.3.1.5 Starting of the HYDRA Word Report

The HYDRA Word Report is started. The file name of the HYDRA Word Report is determined from the first part of the form management entry "file name".
As this file is a Word document template including macros, a new Word document based on this document template (i.e. the HYDRA Word Report) is created by starting it directly (double clicking in the Windows Explorer). The Visual Basic for Application macros defined in the HYDRA Word Report format this document and insert exported HYDRA data.

3.3.2 Valuable information on processing XML files in HYDRA Word Reports

XML files are structured in hierarchies. The terms used in this document are explained in relation with XML data elements in the following example.
The general structure of the example corresponds to the XML files created by MOC in relation with exporting the data contents required for generating the report.

```
<?xml version="1.0" standalone="yes" ?>
<DocumentElement>
  <BOPMaintenanceList>
    <maintenance.classification>A</maintenance.classification>
    <maintenance.type>T</maintenance.type>
```

[1]

```xml
        <maintenance.designation>Inspektionsprüfung</maintenance.designation>
        <maintenance.valid_till>2015-03-18T00:00:00+01:00</maintenance.valid_till>
        <maintenance.active>true</maintenance.active>
        <maintenance.interval type="TIME" unit="YEAR">5</maintenance.interval>
        <maintenance.single_maintenance>false</maintenance.single_maintenance>
    </BOPMaintenanceList>
    <BOPMaintenanceList>
        <maintenance.classification>A</maintenance.classification>
        <maintenance.type>Z</maintenance.type>
        <maintenance.designation>TÜV</maintenance.designation>                      [2]
        <maintenance.valid_till>2012-04-81T00:00:00+01:00</maintenance.valid_till>
        <maintenance.active>true</maintenance.active>
        <maintenance.interval type="PIECE">5</maintenance.interval>
        <maintenance.single_maintenance>false</maintenance.single_maintenance>
    </BOPMaintenanceList>
    <BOPMaintenanceList>
        <maintenance.classification>B</maintenance.classification>
        <maintenance.type>T</maintenance.type>
        <maintenance.designation>Inbetriebnahme</maintenance.designation>           [3]
        <maintenance.valid_till>2099-12-31T00:00:00+01:00</maintenance.valid_till>
        <maintenance.active>true</maintenance.active>
        <maintenance.interval type="TIME" unit="MONTH">32</maintenance.interval>
        <maintenance.single_maintenance>false</maintenance.single_maintenance>
    </BOPMaintenanceList>
</DocumentElement>
```

Legend: XML data record including corresponding data record counter **[1]**

XML data record nodes

Name of the XML data node

Attribute="value"

Value of a data record node

Every entry within the area of an identified XML data record node is described as XML data field node in this document.

The path of an XML data node includes the names of all parent nodes including their indexes and the name of the XML data node itself (including its index). The individual data nodes are separated by slashes (/), the node indexes are put into square brackets.

In the sample structure described here nearly all XML data nodes are addressed with index `1`. This is not the case for XML data record nodes ‚`BOPMaintenanceList`'. In this case the index represents the data record number.

Examples:

- Complete path of the main node of XML data:
  `/DocumentElement[1]`
- Complete path of the first XML data record node of the sample structure:
  `/DocumentElement[1]/BOPMaintenanceList[1]`

- Complete path of the third XML data record node of the sample structure:
  `/DocumentElement[1]/BOPMaintenanceList[3]` :
- Complete path of the XML data field node `maintenance.type` of the third data record of the sample structure:
  `/DocumentElement[1]/BOPMaintenanceList[3]/maintenance.designation[1]`
  The XML data field node referenced in this way includes the value `Inbetriebnahme (implementation).`

The attributes of an XML data node play a minor role in designing a HYDRA Word Report. For this reason, they are not explained in more detail at this point.

The following sections differentiate between XML master data files and XML detail data files.

The general structure of these two XML files is identical.
However, XML master data files only include one data record in the majority of cases. The MOC data export functions that are already described in this document make clear why this is the case and why these two XML file types are distinguished.

3.3.3 Processes after starting a HYDRA Word Report

This paragraph deals with the functions of the macros defined in Word. They are used to integrate exported MOC data and to design the report.

Possible UserExits are also described in this context. They allow for forms to be designed by individual programming.

It generally applies that a UserExit is not executed if Word objects referenced when starting the UserExit (content control, tables, table rows) are deleted by previous actions.

Examples of using these UserExits can be found in a separate section at the end of this document.

3.3.3.1 Automatic start of the AutoMacro DocumentNew

After opening the HYDRA Word Report, Word automatically processes the AutoMacro `DocumentNew()`.
This AutoMacro normally includes first the HYDRA macro library (`hydramacrolibrary.dotm`) that has to be in the same directory as the HYDRA Word Report as document template.
Then the macro `MpdvCreateReport` is started that is included in this library. This macro edits and outputs the report. The paragraphs that follow describe the individual sub-functions processed by this macro in more detail.

3.3.3.2 Preparatory activities

Updating of the screen is disabled at first. This measure increases the performance when editing reports. Moreover, the user is not disturbed by flickering screens.
All required parameters of the print control information (print target, number of copies etc.) are read out of the corresponding XML file and made available to subsequent functions as global variables.

3.3.3.3 Insertion of tabular field information

➔ **UserExit `UeFillTablesFromXmlBefore`**

`oDoc As Word.Document`	→ Reference to the current Word document
`iLoop As Integer`	→ Current document counter
`sXmlDir As String`	→ Directory of HYDRA XML files

At this stage it is checked for all tables included in the HYDRA Word Report whether information from XML detail data files is to be inserted. The procedure is as follows:

It is checked in the alternative text description of the table that is currently being processed whether references to XML detail data files exist. The table is not processed if such references are not found.
In case a reference to a data file is found it is attempted to determine the complete file name. The file extension `.xml` is added if required and the name of the directory from which the HYDRA Word Report was opened is also prefixed, but only if necessary.
A corresponding error message is output if the XML detail data file determined in this way does not exist or the included XML information cannot be interpreted. In these cases as well, the table is not processed.

In case the XML detail data file relevant for the table is found, the automatic column fit is suppressed for the table to prevent the designed column widths from getting lost while processing the table.

The fixed part of the XML data record node name (without the index of the current data record) is also determined from the alternative text description. Its information is to be used for filling out the table.
In case it is an XML master data file generated by HYDRA a `/DocumentElement[1]/` is prefixed to reference the XML data record node.

➔ **UserExit `UeFillTableFromXmlBefore`**

`oDoc As Word.Document`	→ reference to the current Word document
`iLoop As Integer`	→ current document counter
`oTbl As Word.Table`	→ reference to the current table to be processed
`sXmlDataNodeSub As String`	→ Name of the XML data record node (from the alternative text description of the table)
`sXmlFilePath As String`	→ file name of the XML detail data file of the table
`sXmlData As String`	→ XML data of the detail data file as string

The last row of the table (in the following referred to as reference table row) is stored in an AutoText component called `TmpAutoText`_***[accidental character string]*** as copy template for the data records of the XML detail data records to be inserted.
The reference table row is deleted afterwards.

All (indented) steps described in the paragraphs that follow are completed for every data node of the corresponding XML detail data file of the table.

> The reference table row stored in the AutoText component is inserted at the end of the table.
>
> Then all cells of the inserted reference table row are checked whether or not they include further tables. This processing of table information is started recursively for each of these tables. Consequently, additional data can be read in to these nested sub-tables from other XML detail data files, if required.
>
> Now the path of the XML data record node the information of which is to be used to edit the current table row is determined (if necessary by using the index of the current data record).
>
> ➔ **UserExit `UeFillRowFromXmlBefore`**

`oDoc As Word.Document`	→ reference to the current Word document
`iLoop As Integer`	→ current document counter
`oTbl As Word.Table`	→ reference to the current table to be processed
`sXmlDataNodeSub As String`	→ Name of the XML data record node (from the alternative text description of the table)
`sXmlFilePath As String`	→ XML detail data file of the table
`sXmlData As String`	→ XML data of the detail data file as string
`sXmlDataNode As String`	→ Path of the current XML data record node
`oRow As Word.Row`	→ reference to the current row to be processed

Then the below-described activities are performed for all content controls included in the current table row and relevant to XML connections:

If the current text (not the name!) of the content control starts with the string `#LINK:` the content control is considered being relevant for XML connections. In this case, the name of the XML data field node the data of which is later to be included in the content control is determined from the characters behind `#LINK:`.

Optionally, the name of the XML data field node might be followed by formatting instructions (starting with the key term `#FORMAT:`). This instruction is also read out and made available to further processing.

The path of the XML data field node is determined from the name of the XML data field node and the path of the current XML data record node. The content of the XML data field node is read out afterwards. In case there is no XML data field node with this name, the

➔ **UserExit `UeLinkTableCtrlToXmlNotFound`**

`oDoc As Word.Document`	→ reference to the current Word document
`iLoop As Integer`	→ current document counter
`oTbl As Word.Table`	→ reference to the current table to be processed
`sXmlDataNodeSub As String`	→ name of the XML data record node (from alternative text descriptions of the table)
`sXmlFilePath As String`	→ XML detail data file of the table
`sXmlData As String`	→ XML data of detail data file as string
`sXmlDataNode As String`	→ path of the current XML data record node
`oRow As Word.Row`	→ reference to the current row to be processed
`oCc As Word.ContentControl`	→ reference to the current content control
`sCcXmlLink As String`	→ path to the current XML data field node
`sCcValueBefore As String`	→ original text of the content control
`sCcXmlLinkBefore As String`	→ original XML link of the content control (without `#LINK:`)
`sCcFormatBefore As String`	→ format parameter of the content control (without `#FORMAT:`)

is started and processing is continued with the next content control of the current row.

Provided that the content of the XML data field node could be read out, its value is converted automatically into the format of the corresponding Visual Basic for Application data type. The result of this conversion can be accessed, if required, in the below-described UserExit.

➔ **UserExit `UeLinkTableCtrlToXmlBefore`**

`oDoc As Word.Document`	→ reference to the current Word document
`iLoop As Integer`	→ current document counter
`oTbl As Word.Table`	→ reference to the current table to be processed
`sXmlDataNodeSub As String`	→ name of the XML data record node (from alternative text description of the table)
`sXmlFilePath As String`	→ XML detail data file of the table
`sXmlData As String`	→ XML data of the detail data file as string
`sXmlDataNode As String`	→ path of the current XML data record node
`oRow As Word.Row`	→ reference to the current row to be processed
`oCc As Word.ContentControl`	→ reference to the current content control
`sCcXmlLink As String`	→ path of the current XML data field node
`sCcValueBefore As String`	→ original text of the content control
`sCcXmlLinkBefore As String`	→ original XML link of the content control (without `#LINK:`)
`sCcFormatBefore As String`	→ format parameter of the content control (without `#FORMAT:`)
`vSetValue As Variant`	→ result of data type conversion

The result of the data type conversion is formatted accordingly, provided that corresponding formatting has been specified. It might be the case that the content control type is converted from "text only" to "checkbox" or "picture".

In any case, the content control value is replaced by the content of the XML data field node.

In case the option "Remove content control when contents are edited" is set for the content control it will be removed by setting the value. Consequently, the document only includes this value and the following UserExit is no longer run through as the referenced parameter `oCc` is not available anymore.

➔ **UserExit `UeLinkTableCtrlToXmlAfter`**

`oDoc As Word.Document`	→ reference to the current Word document
`iLoop As Integer`	→ current document counter
`oTbl As Word.Table`	→ reference to the current table to be processed
`sXmlDataNodeSub As String`	→ name of the XML data record node (from alternative text description of the table)
`sXmlFilePath As String`	→ XML detail data file of the table
`sXmlData As String`	→ XML data of the detail data file as string
`sXmlDataNode As String`	→ path of the current XML data record node
`oRow As Word.Row`	→ reference to the current row to be processed
`oCc As Word.ContentControl`	→ reference to the current content control
`sCcXmlLink As String`	→ path of the current XML data field node
`sCcValueBefore As String`	→ original text of the content control
`sCcXmlLinkBefore As String`	→ original XML link of the content control (without `#LINK:`)
`sCcFormatBefore As String`	→ format parameter of the content control (without `#FORMAT:`)
`sCcValueAfter As String`	→ current content of the content control

The below-described UserExit is started, once all content controls of the current row have been processed.

➔ **UserExit `UeFillRowFromXmlAfter`**

`oDoc As Word.Document`	→ reference to the current Word document
`iLoop As Integer`	→ current document counter
`oTbl As Word.Table`	→ reference to the current table to be processed
`sXmlDataNodeSub As String`	→ name of the XML data record node (from alternative text description of the table)
`sXmlFilePath As String`	→ XML detail data file of the table
`sXmlData As String`	→ XML data of the detail data file as string
`sXmlDataNode As String`	→ path of the current XML data record node
`oRow As Word.Row`	→ reference to the current row to be processed

The previously generated AutoText component representing the reference table row is deleted, once all XML data nodes have been inserted for the current table.

➔ **UserExit `UeFillTableFromXmlAfter`**

`oDoc As Word.Document`	→ reference to the current Word document
`iLoop As Integer`	→ current document counter
`oTbl As Word.Table`	→ reference to the current table to be processed
`sXmlDataNodeSub As String`	→ name of the XML data record node (from the alternative text description of the table)
`sXmlFilePath As String`	→ file name of the XML detail data file of the table
`sXmlData As String`	→ XML data of the detail data file as string

Now the report should include all contents of XML detail data files, once these processing steps have been performed for all tables (including nested tables).

3.3.3.4 Insertion of simple field information

➔ **UserExit `UeLinkCtrlElemToXmlBefore`**

`oDoc As Word.Document`	→ reference to the current Word document
`iLoop As Integer`	→ current document counter
`sXmlDir As String`	→ directory of HYDRA XML files

This processing step connects all remaining content controls of the current document with data nodes of corresponding XML master data file(s).

Document properties called `HydraXmlMasterFile_`***[number 1 to 10]*** are read out to determine the file name of XML master data file(s).

The content is read out for each document property (10 at most). This content is used to determine the file name of the corresponding XML master data file. The file extension `.xml` is added if required and the name of the directory from which the HYDRA Word Report was opened is also prefixed, but only if necessary.
A corresponding error message is output if no file is found with this name or its XML contents cannot be interpreted.

All content controls relevant for XML connections of the current document are attempted to be connected with one of the defined master data files.

If the current text (not the name!) of the content control starts with the string #LINK: the content control is considered being relevant for XML connections. In this case, the name of the XML data field node the data of which is later to be included in the content control is determined from the characters behind `#LINK:`.

Optionally, the name of the XML data field node might be followed by formatting instructions (starting with the key term `#FORMAT:`). This instruction is also read out and made available to further processing.

If this is an XML master data file generated by HYDRA `/DocumentElement[1]/` is prefixed to the path of the XML data field node. The content of the XML data field node is read out afterwards. In case there is no XML data field node with this name, the

➔ **UserExit `UeLinkMasterCtrlToXmlNotFound`**

`oDoc As Word.Document`	→ reference to the current Word document
`iLoop As Integer`	→ current document counter
`sXmlFilePath As String`	→ current XML master data file
`sXmlData  As String`	→ XML data of the current master data file as string
`oCc As Word.ContentControl`	→ reference to the current content control
`sCcXmlLink As String`	→ path of the current XML data field node
`sCcValueBefore As String`	→ original text of the content control
`sCcXmlLinkBefore As String`	→ original XML link of the content control (without `#LINK:`)
`sCcFormatBefore As String`	→ format parameter of the content control (without `#FORMAT:`)

is started and processing is continued with the next content control.

Provided that the content of the XML data field node could be read out, its value is automatically converted into the format of the corresponding Visual Basic for Application data type. The result of this conversion can be accessed, if required, in the below-described UserExit.

➔ **UserExit `UeLinkMasterCtrlToXmlBefore`**

`oDoc As Word.Document`	→ reference to the current Word document
`iLoop As Integer`	→ current document counter
`sXmlFilePath As String`	→ current XML master data file
`sXmlData  As String`	→ XML data of the current master data file as string
`oCc As Word.ContentControl`	→ reference to the current content control
`sCcXmlLink As String`	→ path of the current XML data field node
`sCcValueBefore As String`	→ original text of the content control
`sCcXmlLinkBefore As String`	→ original XML link of the content control (without `#LINK:`)
`sCcFormatBefore As String`	→ format parameter of the content control (without `#FORMAT:`)
`vSetValue As Variant`	→ result of the data type conversion

The result of the data type conversion is formatted accordingly, provided that corresponding formatting has been specified. It might be the case that the content control type is converted from "text only" to "checkbox" or "picture".

In any case, the content control value is replaced by the content of the XML data field node.

In case the option "Remove content control when contents are edited" is set for the content control it will be removed by setting the value. Consequently, the document only includes this value and the following UserExit is no longer executed as the referenced parameter `oCc` is not available anymore.

➔ **UserExit `UeLinkMasterCtrlToXmlAfter`**

`oDoc As Word.Document`	→ reference to the current Word document
`iLoop As Integer`	→ current document counter
`sXmlFilePath As String`	→ current XML master data file
`sXmlData  As String`	→ XML data of the current master data file as string
`oCc As Word.ContentControl`	→ reference to current content control
`sCcXmlLink As String`	→ path of the current XML data field node
`sCcValueBefore As String`	→ original text of the content control
`sCcXmlLinkBefore As String`	→ original XML link of the content control (without `#LINK:`)
`sCcFormatBefore As String`	→ format parameter of the content control (without `#FORMAT:`)
`sCcValueAfter As String`	→ current content of the content control

If all content controls of the document are processed, the following UserExit is started.

➔ **UserExit `UeLinkCtrlElemToXmlAfter`**

`oDoc As Word.Document` → reference to the current Word document

`iLoop As Integer` → current document counter

`sXmlDir As String` → directory of HYDRA XML files

3.3.3.5 Final activities and output of the HYDRA Word Report

Once all data from XML files has been integrated in the HYDRA Word Report, the text of the remaining control elements relevant to XML connections that have not yet been connected is emptied. It is deleted if the checkbox "Remove content control when contents are edited" is set. Otherwise, its placeholder text is removed.
This reworking of content controls that have not been connected is suppressed if the current document includes the property `HydraSuppressCcCleaning` (of the type `yes or no`) assigned to the value `yes`.

➔ **UserExit `UeOutputReportBefore`**

`oDoc As Word.Document` → reference to the current Word document

`iLoop As Integer` → current document counter

`sXmlDir As String` → directory of HYDRA XML files

At this point the view is switched to page layout and the cursor is set to the beginning of the document. Then updating of the screen is again enabled.

Now the modification flags of the current document and the corresponding document template are reset. Consequently, no confirmation prompt asking whether or not the changes are to be saved appears when closing the document.

Finally, the generated document is output as specified and archived if this has been configured beforehand.

3.4 Design of a HYDRA Word Report

3.4.1 Create a new HYDRA Word Report

A new document template is to be prepared if a new HYDRA Word Report is to be created. The procedure is described in the paragraphs that follow.

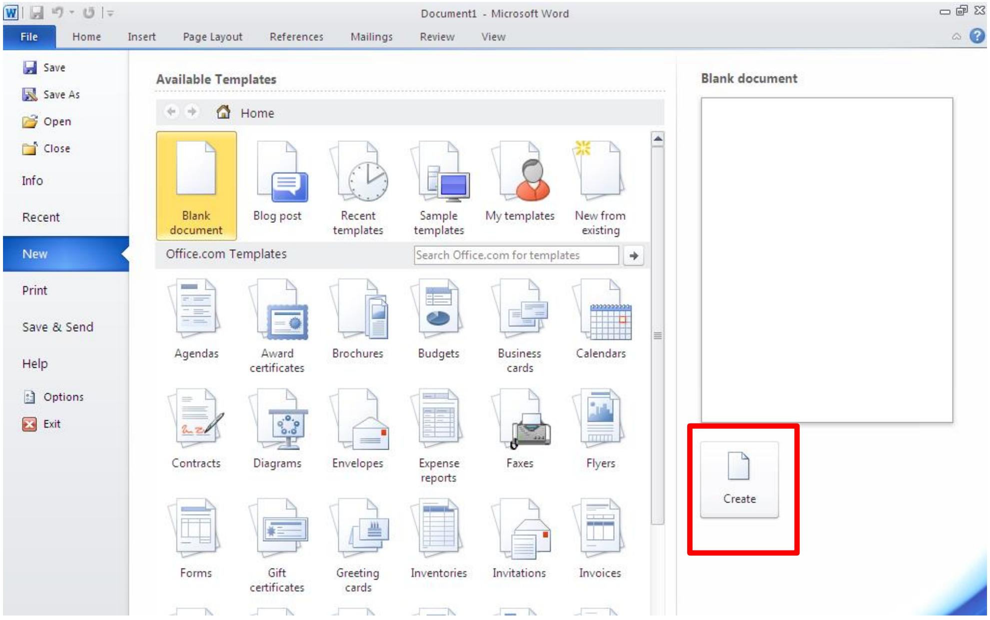
Please note: As long as you are designing a report you should prefix an underscore to the file name of the Word Report.

This prevents the local draft of the HYDRA Word Report from being overwritten by the HYDRA server version when starting the HYDRA Word Report from the print dialog.

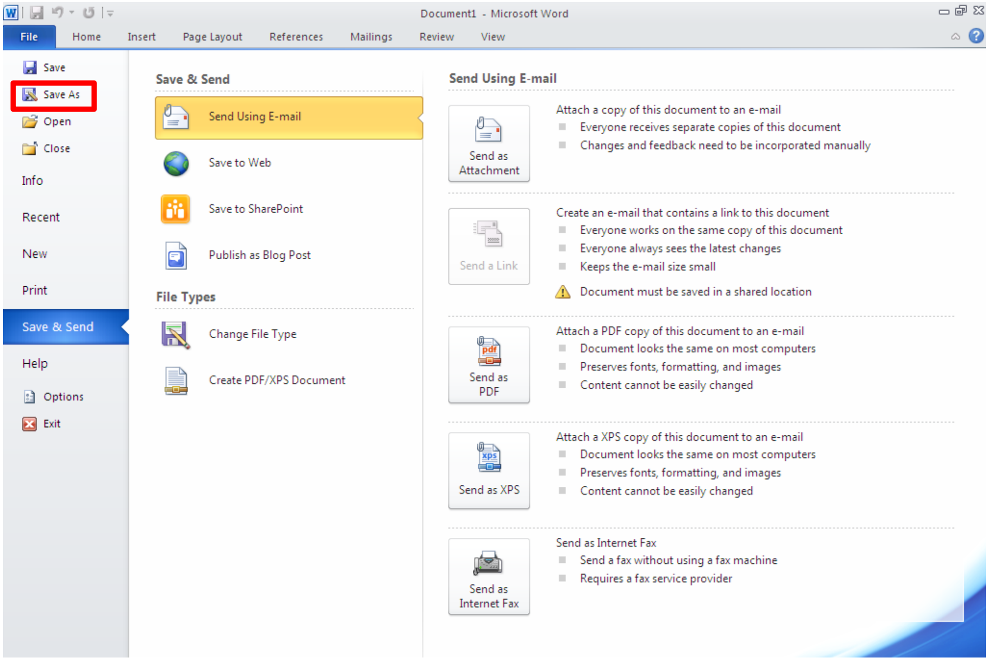
When the HYDRA Word Report is finally started it is verified whether there is file for the selected Word Report that includes an underscore. If this is the case, this file is started instead of the server version. Only in case such a document does not exist, the report loaded by the server is opened.

As an alternative to the manual creation, an existing HYDRA Word Report may also be copied under a new name. In this case, the steps described in this paragraph can be dropped.

1. Generate a new Word document.

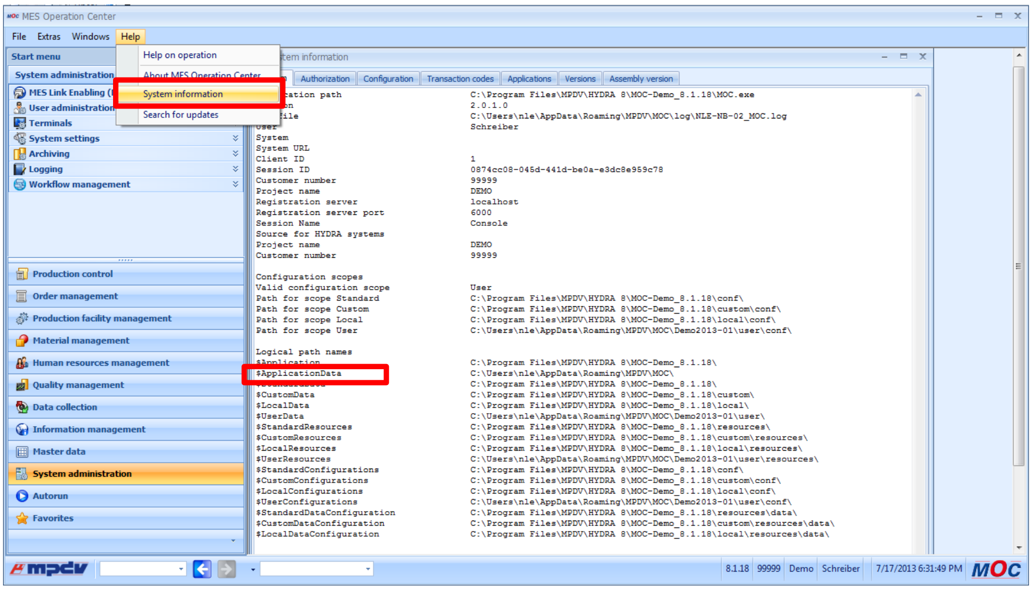


2. Save the new document



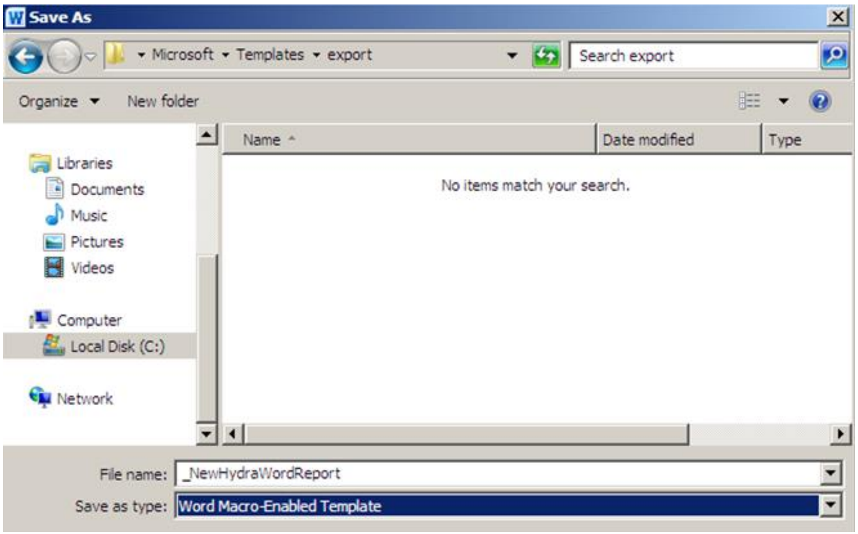
Please use the subfolder `export` of the MOC user data directory.

Please note: The MOC user data directory can be determined by reading out the content of the parameter `$ApplicationData` in the MOC Help menu -> System information.

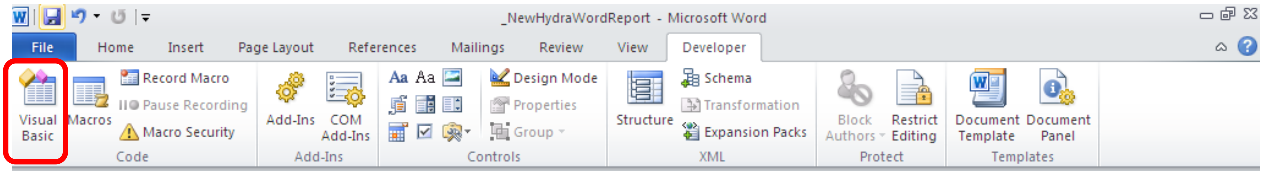


Choose a file name that corresponds to the form management configuration in Master data → Forms. As described before, prefix an underscore to the defined file name.

Choose Word Macro-Enabled Template as file type.

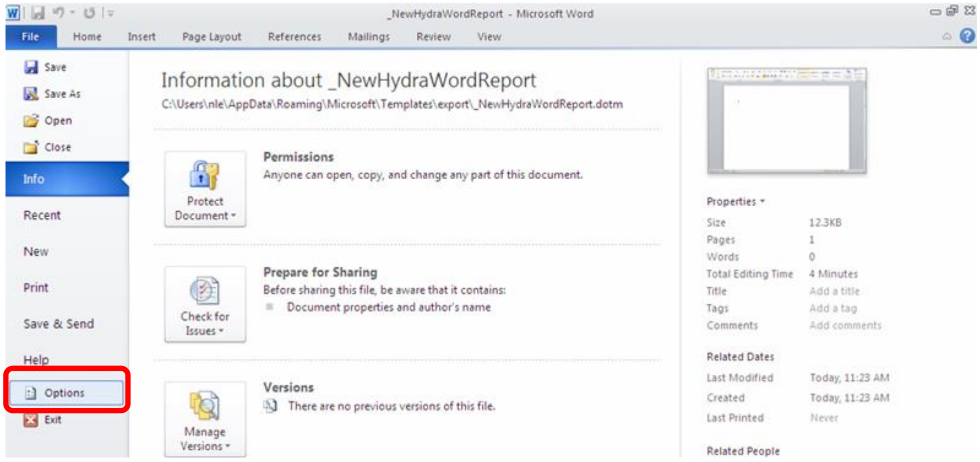


3. Open the dialog to manage Visual Basic for Applications in Word.

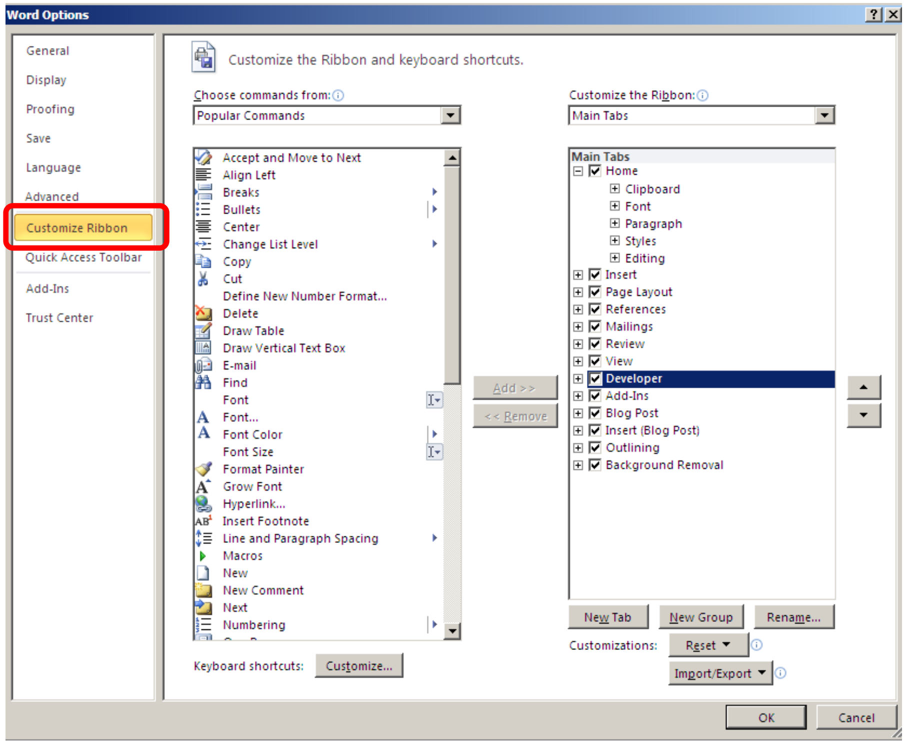


Please note: The main tab "developer" has to be enabled in Word to select the corresponding buttons.

This can be configured in the dialog window File → Options

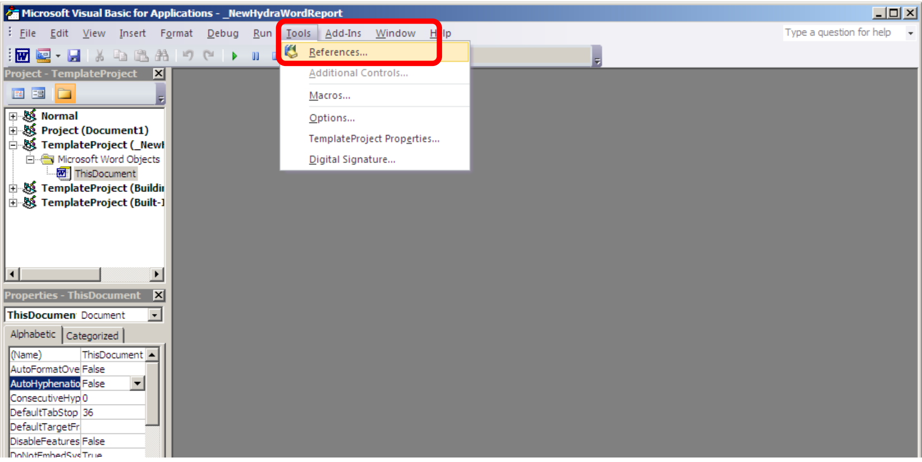


by checking the "Developer Tools" entry in the "Customize Ribbon“ option.

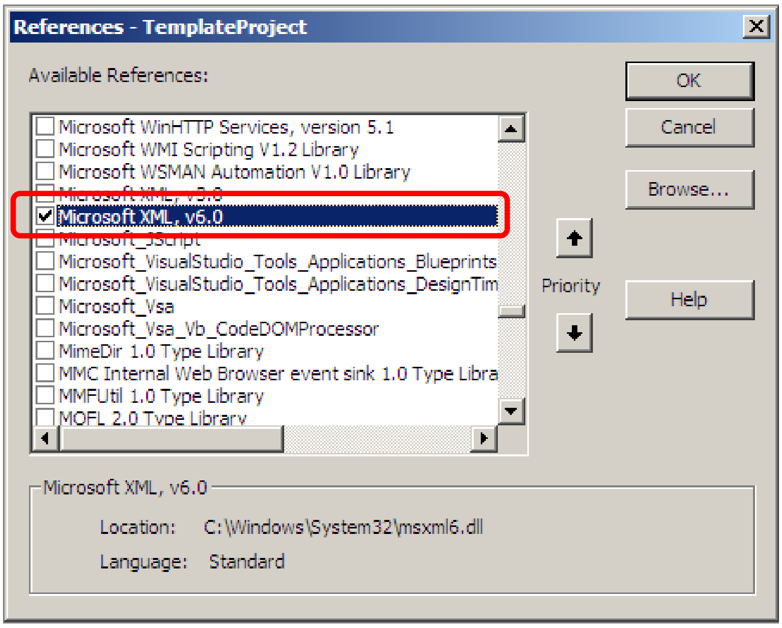


4. Activate the reference for XML file processing.

   To do so, choose the "References" option in the menu item Tools.

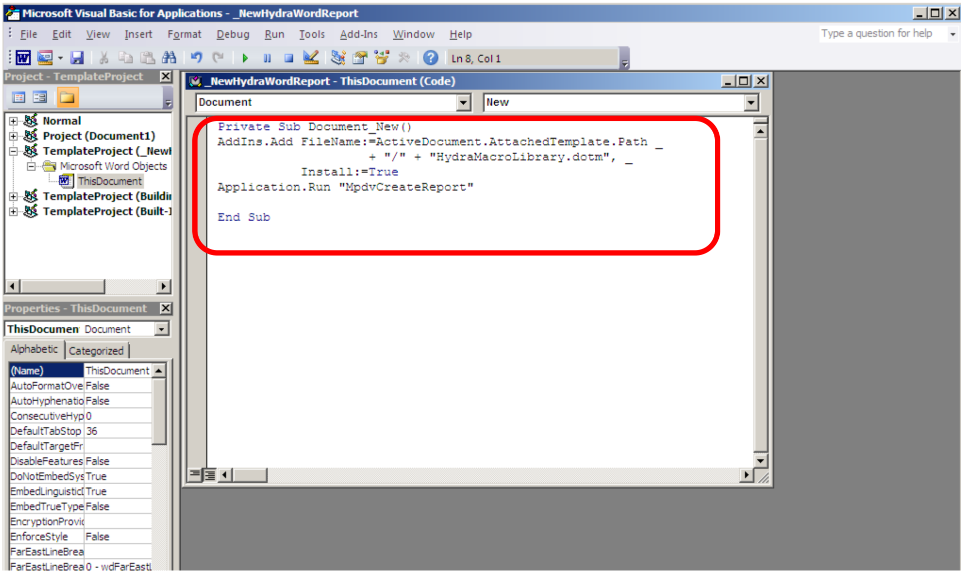


   Enable the entry "Microsoft XML, V6.0“ in the dialog that opens.



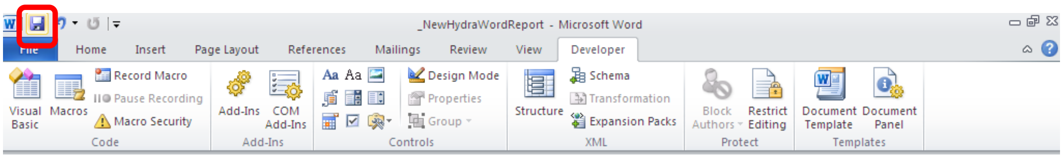
5. Create the Word AutoMakro `Document_New()` by inserting the below text for your current document template in the `ThisDocument` branch:

```
Private Sub Document_New()

    AddIns.Add FileName:=ActiveDocument.AttachedTemplate.Path _
                         + "/" + "HydraMacroLibrary.dotm", _
               Install:=True
    Application.Run "MpdvCreateReport"

End Sub
```



The dialog to manage Visual Basic for Applications can now be closed.

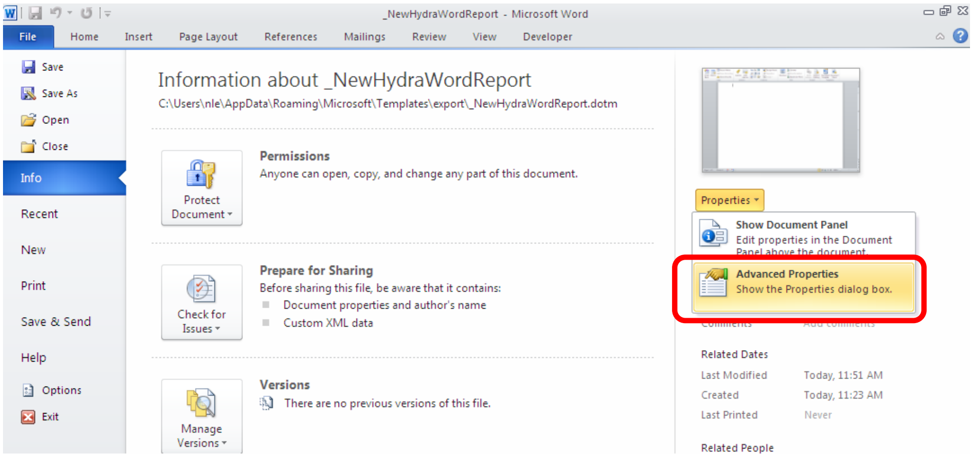
6. Save your changes



You have created a new (but empty) HYDRA Word Report that can be used as basis for designing the report.

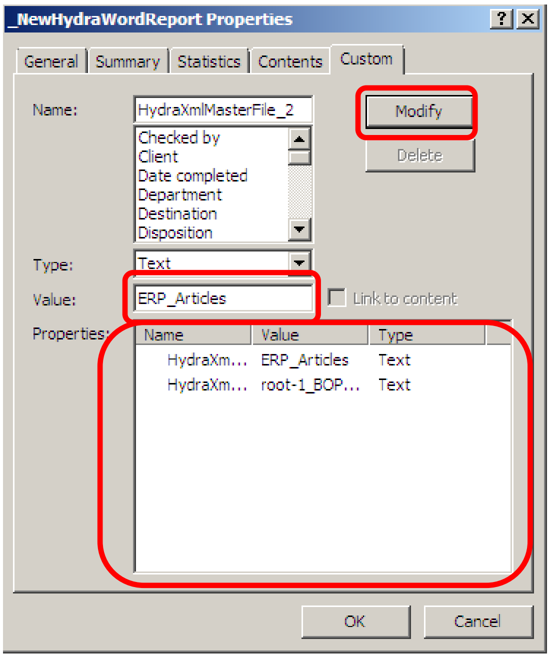
3.4.2 Insertion of simple field information from HYDRA data

At first it has to be defined from which XML master data files simple field information has to be inserted. A maximum of 10 such XML master data files can be referenced by default.
To do so, the button Properties→ Extended Properties is to be clicked in the File --> Information menu of the HYDRA Word Report.

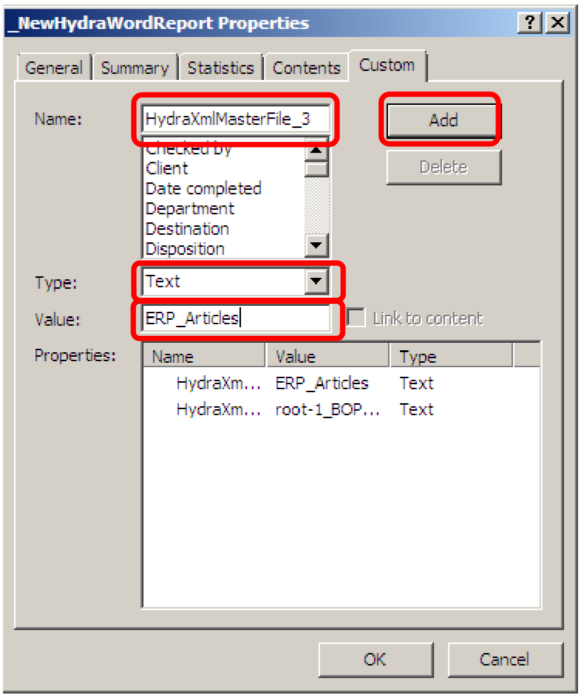


All references to XML master data files are entered in the dialog that opens. A reference to an XML master data file is characterized by a name and value. The name structure is `HydraXmlMasterFile_`***[Index from 1 to 10]***. The value includes the file name of the XML master data file.

To change an existing reference, choose it from the properties list in the lower section of the dialog. Then its value (file name of the XML master data file) can be adjusted. This process is completed by clicking the "modify" button.



To add a new reference, enter its name (according to the above-mentioned structure) in the field of the same name. Make sure that the "text" type is selected. Now enter the file name of the XML master data file as value in the field. Your input is completed by clicking the "add" button and the new document property is now included in the list within the lower section of the dialog.

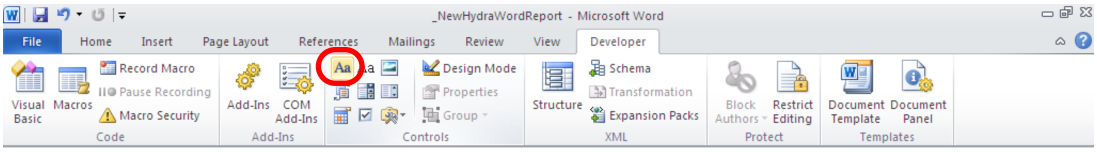


Please note: If the XML master data file is stored in the reporting directory (the same directory that also includes the HYDRA Word Report; this is the default case for XML master data files generated by HYDRA) the directory name does not have to be indicated.

Provided that the XML master data file is provided with the `xml` extension you do not need to enter it either.

Placeholders for simple field information are defined as content controls in the HYDRA Word Report. When reports are edited later, they are connected with the corresponding XML data field node. Consequently, the content control is replaced by the content of the corresponding data field node.

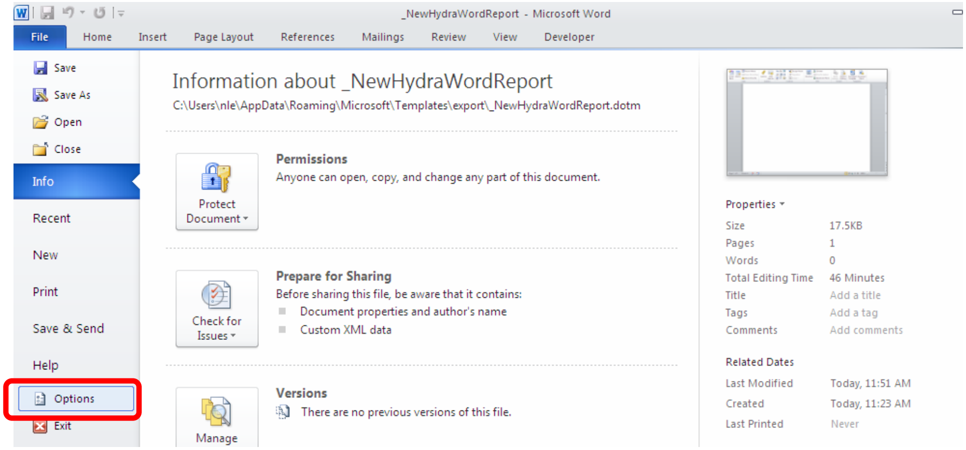
In Word content controls are inserted using the buttons of the "developer" tab.



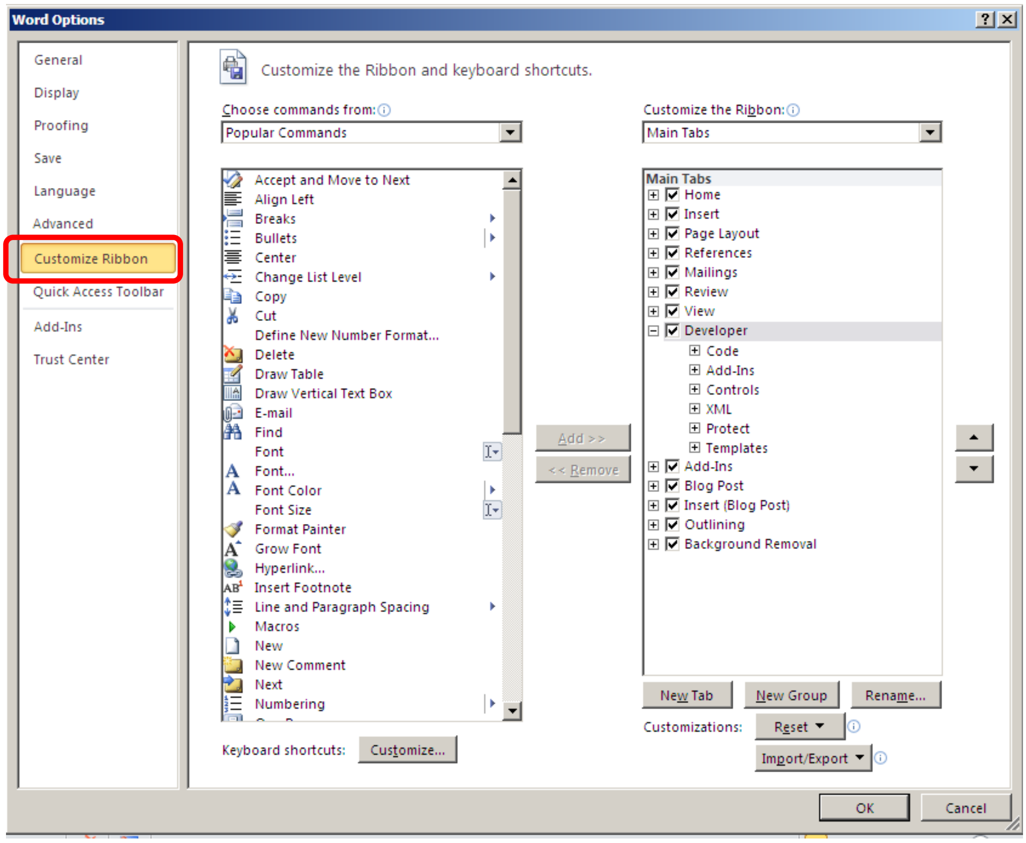
The standard functions of the HYDRA Word Report only support Content Control Text.

Please note: The main tab "developer" has to be enabled first to access the developer tools.

This can be configured in the dialog window File → Options



by checking the "developer" entry in the "Customize Ribbon“ option.



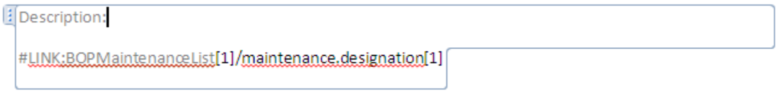
The text of the inserted content control has to include the following pieces of information to make sure that the content of the XML data field node actually appears in the report:

1. <u>Characterizing the content control as relevant to XML connections</u>

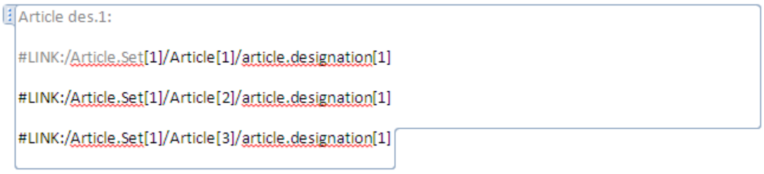
   This is realized by entering the string `#LINK:`

2. <u>Definition of the reference to the XML data field node</u>

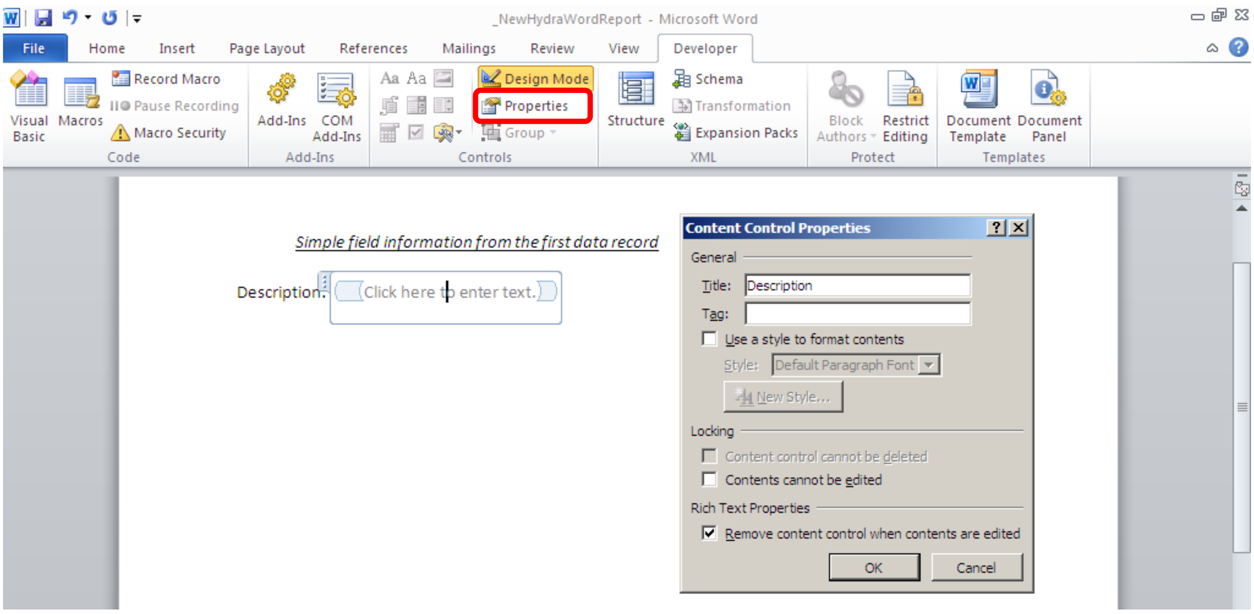
   If the XML master data file is generated by HYDRA the name of the data record node is defined with index (reference to the data record pertaining to it) and the name of the data field node is defined with index (in the majority of cases `1`). Both components are separated by a slash.



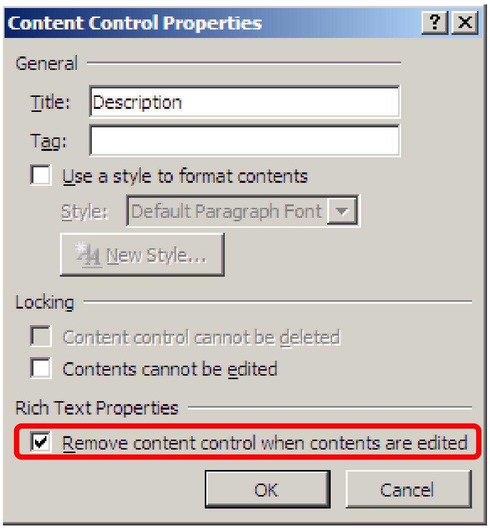
   Provided that this data file is not generated by HYDRA, the entire path of the data record node to be referenced (starting with slash) is to be indicated.



Further details for the content control field can be parameterized in the "developer" tab by clicking the "properties" button. The cursor has to be within the content control that is to be configured to be able to select this button.

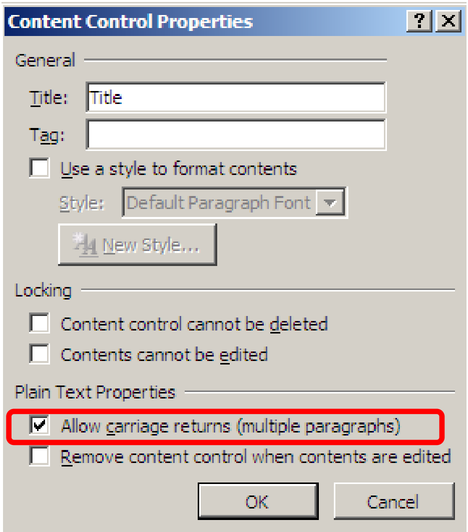


The checkbox "Remove content control when contents are edited" is quite important.

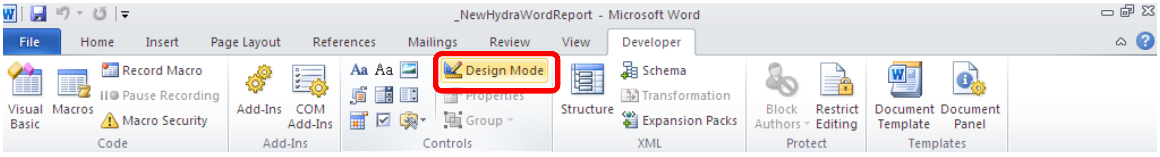


If this option is enabled the content control is removed, once it has been provided with data automatically (or manually). Consequently, the document only includes the inserted text. It is up to the user's decision to enable this element. But in the majority of cases it has a positive effect on the performance when reports are edited at a later point in time. This is especially the case with complex reports that include much data.
Please note that this element may only be activated when designing a HYDRA Word Report, once the reference to the data field node has been entered (as described below).

In case the above referenced XML data field node might include multi-line information the "allow carriage return (several paragraphs)" checkbox should be checked.



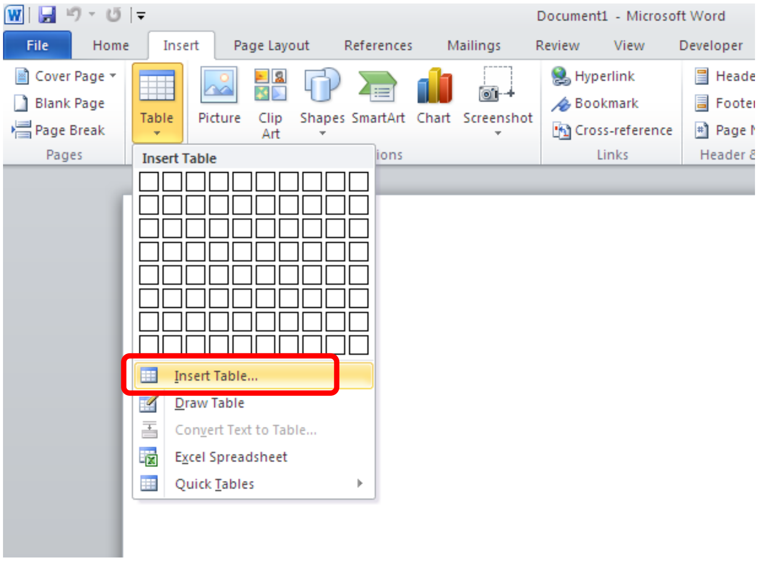
Please note: By activating the “design mode”, you can get a quick overview of all content controls included in a Word Report.



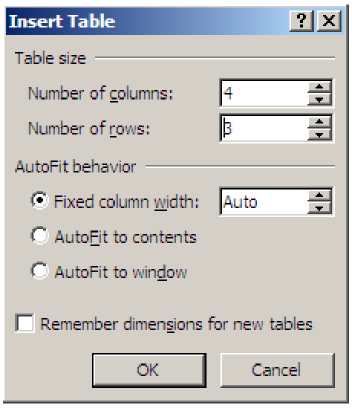
All areas of a HYDRA Word Report (headers, footers, document area) allow for simple field information to be inserted.

3.4.3 Insertion of table information from HYDRA data

The functions defined in the HYDRA macro library help users insert XML detail data files at the end of a table. To do so, create a new table.



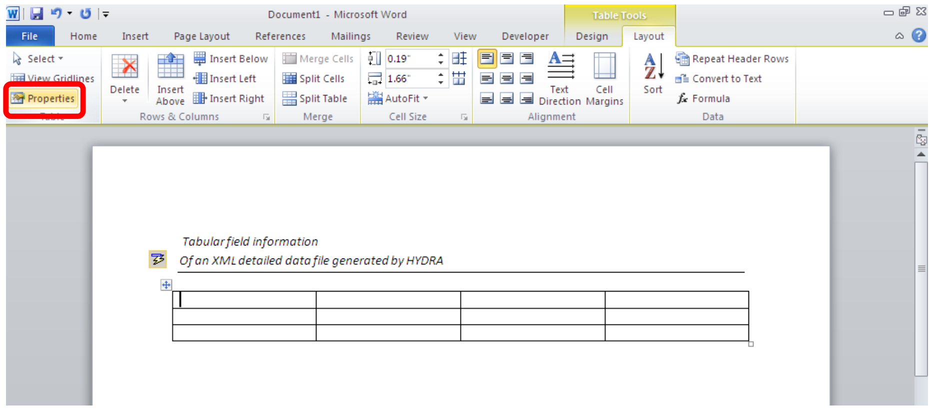
In general, its layout should correspond to the print result that is expected. The last row of the inserted table is reserved for defining the layout of detail data records that are to be inserted at a later point in time. Consequently, this last table row represents a reference to every data record to be inserted from the XML detail data file.



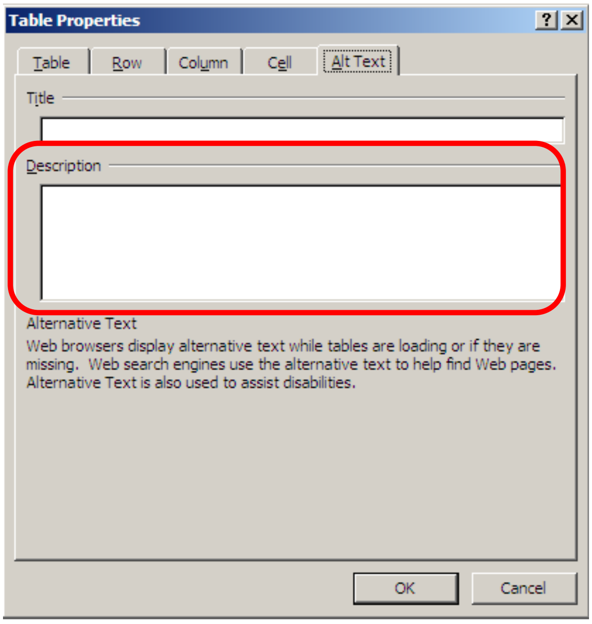
<u>Please note:</u> A blank line has to be inserted below the table to provide sufficient space for the functions to insert new data rows. Otherwise, problems might arise when the report is edited.

The XML data file from which the table is to receive its data has to be referenced. In addition to this, the name or path of the corresponding XML data record node is parameterized.

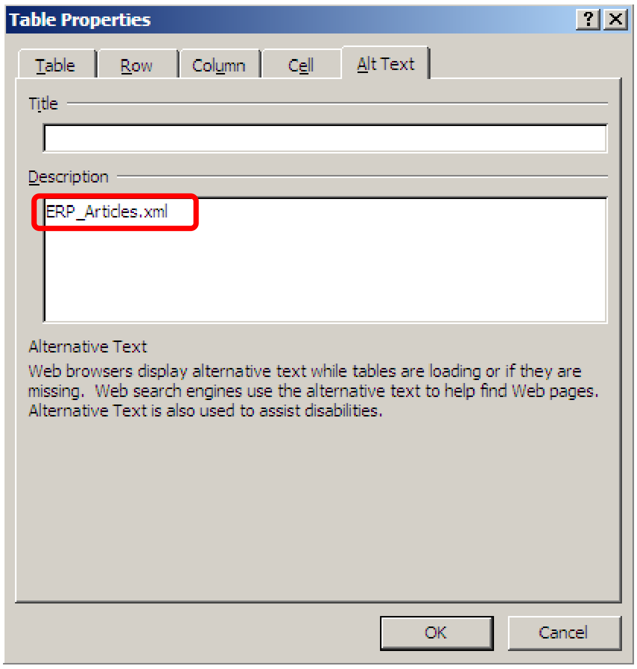
To do this configuration, the cursor is moved in the table and the button Table tools → Table layout → Properties is clicked.



In the "alt text" tab of the dialog that opens the "description" field is used to configure the previously mentioned references .



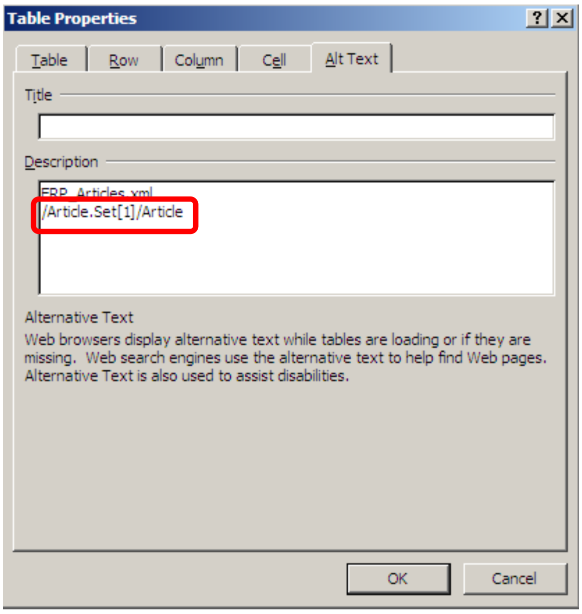
The reference to the name of the file including the data to be inserted is defined in the first row of this field. This file name may include placeholders, which are also described in more detail in this document.



Please note: If the XML master data file is filed in the reporting directory (the same directory that also includes the HYDRA Word Report; this is the default case for XML master data files generated by HYDRA) the directory name does not have to be indicated.

Provided that the XML master data file is provided with the `xml` extension you do not need to enter it either.

The name or path of the XML data record node from the data field nodes of which the information is to be inserted is defined in the second row of the "description" field.



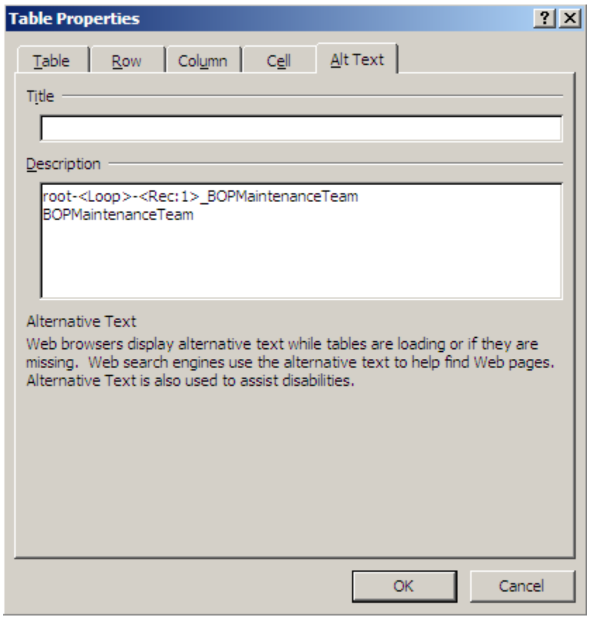
It is sufficient to enter the name when indicating the reference to the corresponding XML data record node, provided that the XML detail data file has been exported by HYDRA (e.g. BOPMaintenanceTeam).

Provided that the XML detail data file derives from another source, the entire path of the data record node starting with a slash has to be specified (e.g. /Article.Set[1]/Article).

The index of the data record node must not be indicated in either case.

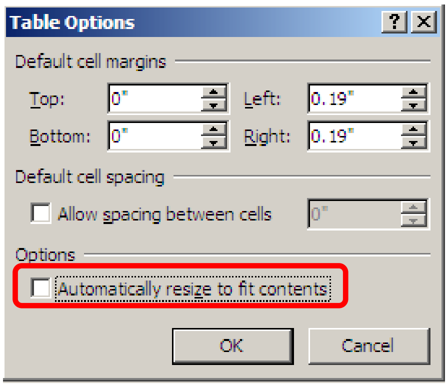
Placeholders may be included in the file name as well as in the reference to the data record node, which resolve the current position within the corresponding hierarchy of data records when reports are edited later. The below placeholders are supported by default:

- <Loop> Current document counter
- <Rec: ***[Hierarchy level]***> reference from the current data record counter of the relevant hierarchy level

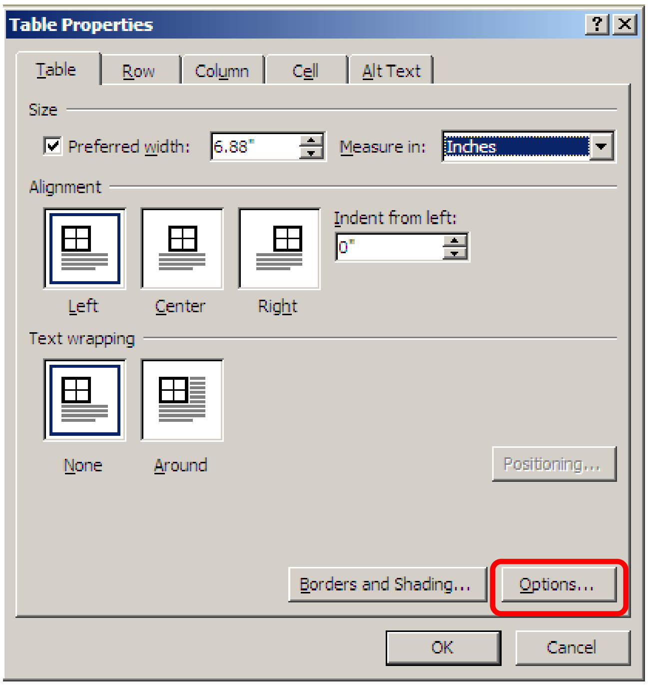


The configuration of the XML data file and the relevant data record node has been completed.

Please note: Once the dialog to configure table properties has been opened, the option "Automatically resize to fit contents" should directly be disabled.



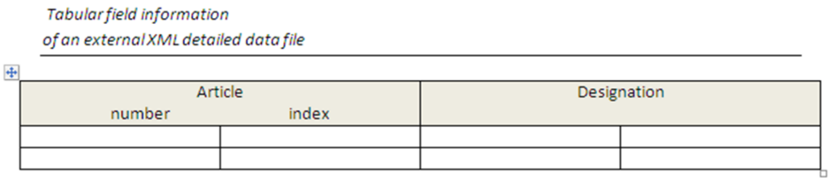
These settings can be opened by clicking the "options" button in the "table" tab.



Problems might arise if this table property remains active as columns might be moved dynamically by inserting texts and later by inserting data from XML detail data nodes. This might alter the specified column layout.

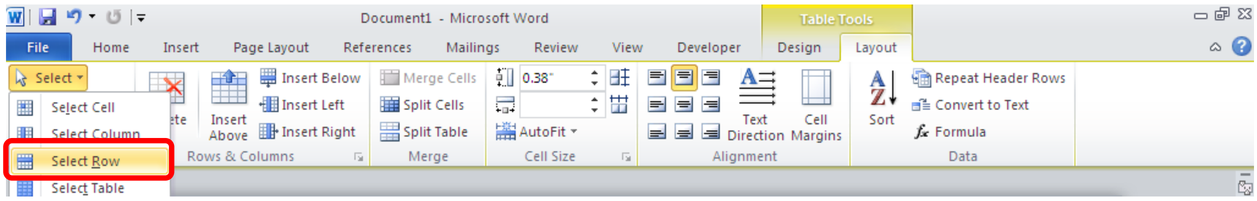
Then the table header can be designed as it is to look like later in the report. In this case special attention should be paid when merging cells as problems might arise when the last table row is selected when reports are edited afterwards.

Whether or not the table header meets the requirements for editing the report can be checked by putting the cursor in the first cell of the last table row (which is still empty).

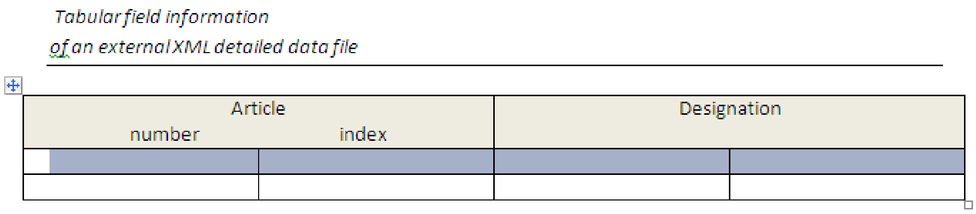

Tabular field information
of an external XML detailed data file

Article		Designation	
number	index		

Now choose the "select row" option in the ribbon "table tools" --> "layout" --> "select".



Problems are unlikely to arise while editing the report later if cells of the last row are selected only.


Tabular field information
of an external XML detailed data file

Article		Designation	
number	index		

As already mentioned, the last table row is used as reference design for the data records to be inserted at a later point in time.

A corresponding content control is inserted at each position where information from data field nodes of the XML detail data file is to be inserted. This has already been described in more detail in the section dealing with simple field information.

The decisive difference to simple field information is that the text of the corresponding content control only includes the name of the data element node (without index). The name of the data record node is no longer important as it has already been referenced within the table properties.

Tabular field information
of an external XML detailed data file

Article		Designation	
number	index		
#LINK:article.id	#LINK:article.index	Article designation #LINK:article.designation	

The number and alignment of content controls is not restricted and may be configured according to the user's requirements in the cells of the last table row.

Table information may be inserted in all areas of a HYDRA Word Report (headers, footers, document area).

3.4.4 Nesting of table information

Further sub-tables can be inserted within a table cell, if required. The cell into which a sub-table is to be inserted has to be included in the previously described reference row.

Optionally, the header information may be dropped for this sub-table. Consequently, it only includes the reference row of the sub-table. In this case, the sub-table will be deleted when the report is structured afterwards and if no detail data records to be inserted into the corresponding XML detail data file can be found.

A nested table is inserted by going with the cursor to that cell of the reference row in which the sub-table is to be inserted. This table can be inserted and formatted as it is described in the previous section.

Nested tabular field information
of an external XML detailed data file

Article		Supplier	
number	index	number	name
#LINK:article.id	#LINK:article.index	#LINK:supplier.id	#LINK:supplier.name

If required, further sub-tables may be nested within these sub-tables, provided that the previously-mentioned notes are respected.

3.4.5 Formatting of inserted field information

3.4.5.1 Determination of the data type

The information read from XML files is automatically converted into a corresponding data type (date/time, numbers, logical value, character string). This conversion is based on the identification of specific samples for different data types.
In case this automatic conversion fails data has to be edited manually by using programming interfaces, if required.

The automatic conversion into the corresponding data types allows for the below-described formatting options to be used for every inserted field detail.

3.4.5.2 Formatting of field contents

A formatting parameter may optionally be added to a reference to an XML data element node. This parameter starts with the `#FORMAT:` string and is directly added to the reference of the data element node.

Formatting of date/time field contents using the example of a data record of an XML master data file generated by HYDRA

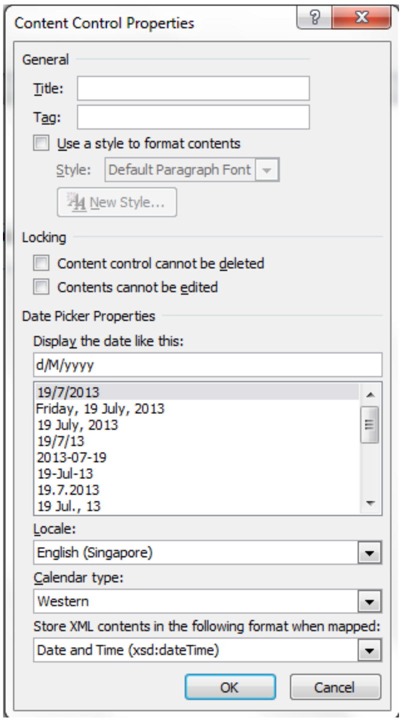
Dat :Date
#LINK:BOPMaintenanceList[1]/maintenance.valid_till[1]#FORMAT:dd/mm/yy

Basically, three formatting types can be distinguished:

1. User-friendly text formatting within the content control
   This formatting type only formats the text of the Content Control Text used in the design.
   The formatting parameters used here correspond to the FORMAT functions known from Visual Basic for Applications. For further information on this function please refer to the help files or relevant literature.
2. Enhanced formatting of text included in the content control
   The above-described formatting options provided by Visual Basic for Applications have been enhanced by further special formatting options.
3. This formatting option converts the content control type that was of the "text only" type when designing the report and adjusts it to the corresponding data type.
   By changing the type of the content control, the user is provided with specialized functions, among other things, e.g. to change a date or logical value at a later point in time.

3.4.5.3 Formatting of date/time information

Format	Type	Description
`dd/mm/yy`	1	Date with days, months and years including two characters
`dd-mmmm-yyyy`	1	Date with month names
`hh:nn:ss`	1	Time
`dd-mm-yyyy h:n:s`	1	Date and time
`General Date`	1	Shows a date and/or time (depending on the content of the element). System settings define how the date and time are displayed.
`Long Date`	1	Shows the date in the long date format according to system settings.
`Medium Date`	1	Shows a date in the medium date format determined by the language version.
`Short Date`	1	Shows the date in the short date format according to system settings.
`Long Time`	1	Shows the time according to the settings for the long time format including hours, minutes and seconds.
`Medium Time`	1	Shows a time in the 12 hours format with hours, minutes and the AM/PM ID.
`Short Time`	1	Shows a time in the 24 hours format.

`DATE_CC:yyyy-MM-dd`	3	The content control of the "text only" type is converted into the "date picker" type. The format in which the date is to be displayed in the content control for date selection is to be entered behind `DATE_CC:`.  The available formats can be viewed by creating a temporary content control of this type and displaying its properties.

3.4.5.4 Formatting of figures

Zeros are not taken into account if numeric values are formatted. However, as described in one of the below examples, positive and negative figures can be formatted differently.

Format	Type	Description
`######,##0.00`	1	Displays thousands separator and two fixed decimal places
`######,##0.0##`	1	Displays thousands separator and one fixed decimal place but a maximum of three decimal places
`########0`	1	Displays no thousands separator and no decimal places

`###0;(###0)`	1	Displays no thousands separator, with alternative format for negative figures
`General Number`	1	Shows the number without thousands separator.
`Currency`	1	Shows the currency according to the system settings for the locale.
`Fixed`	1	Shows at least one place to the left and two places to the right of the decimal separator.
`Standard`	1	Shows the figure with thousands separator as well as at least one place to the left and two places to the right of the decimal separator.
`Percent`	1	Shows the number multiplied by 100 and a percentage (%) added to the right. The figure always has two decimal places.
`Scientific`	1	Uses the scientific standard format.
`DecPlaces`	1	Shows the figure with the number of decimal places specified in the corresponding XML file in the qmcharacteristic.decimal_places data element node. Two decimal places are displayed if no value is indicated or the data element node does not exist.

3.4.5.5 Formatting of logical information

Format	**Type**	**Description**	
		Output if true	**Output if false**
`True/False`	1	`True`	`False`
`Yes/No`	1	`Yes`	`No`
`On/Off`	1	`On`	`Off`
`X/`	2	`X`	
`x/`	2	`x`	

<table>
<tr><td rowspan="2">Symbol:1</td><td rowspan="2">3</td><td colspan="2">The content control of the "text only" type is converted into the "checkbox" type.</td></tr>
<tr><td>✓</td><td>✖</td></tr>
<tr><td rowspan="2">Symbol:2</td><td rowspan="2">3</td><td colspan="2">The content control of the "text only" type is converted into the "checkbox" type.</td></tr>
<tr><td>☑</td><td>☒</td></tr>
<tr><td rowspan="2">Symbol:3</td><td rowspan="2">3</td><td colspan="2">The content control of the "text only" type is converted into the "checkbox" type.</td></tr>
<tr><td>☑</td><td>☐</td></tr>
</table>

3.4.5.6 Formatting of graphics

Format	Type	Description
PICTURE	3	The content of the data node element is interpreted as link to a graphic file. If it is only a file name the directory of HYDRA XML data is prefixed. Then the content control of the "text only" type is converted in to the "picture" type and the linked graphic file is inserted. The size of the graphic is not adjusted in this case and, as a result, determined by the Word functions in use.
PICTURE:400	3	The content of the data node element is interpreted as link to a graphic file. If it is only a file name the directory of HYDRA XML data is prefixed. Then the content control of the "text only" type is converted in to the "picture" type and the linked graphic file is inserted. The graphic width in pixels can be determined by the parameter behind the colon (in this case 400). The picture is automatically increased or reduced to this width, whereas the aspect ratio of the graphic remains.

PICTURE:150MAX	3	The content of the data node element is interpreted as link to a graphic file. If it is only a file name the directory of HYDRA XML data is prefixed. Then the content control of the "text only" type is converted into the "picture" type and the linked graphic file is inserted. The maximum graphic width in pixels can be determined by the parameter that is followed by a MAX value behind the colon (in this case 150). The picture is only reduced automatically to this width if it exceeds this size. The aspect ratio of the graphic remains.

3.4.6 Enhancement of the Word Report by individual programming

The above-described UserExits are available for implementing individual VBA program elements to design HYDRA Word Reports.

To create such a UserExit go to "Visual Basic" in the "developer tools" tab. The corresponding editor including the project overview opens in the left section. Add a new module in the "module" sub-menu below the opened document by "insert" --> "module" and insert the name of the UserExit you would like to use in the properties dialog.

The below examples still require a support function returning the text of an XML data field node. Insert this support function above the first, individual UserExit.

Support function: Text from XML data field node

```
Public Function GetXmlNodeText(sXmlData As String, sFullDataNodeName As String, _
                                   Optional sDefaultText As String = "") As String
    Dim oCxp As Office.CustomXMLPart
    Dim oXmlNode As Office.CustomXMLNode

    GetXmlNodeText = sDefaultText

    Set oCxp = ActiveDocument.CustomXMLParts.Add
    oCxp.LoadXML sXmlData
    Set oXmlNode = oCxp.SelectSingleNode(sFullDataNodeName)

    If Not oXmlNode Is Nothing Then
        GetXmlNodeText = oXmlNode.Text
     End If
End function
```

Example: Table sorting

```
Sub Main(oDoc As Word.Document, iLoop As Integer, oTbl As Word.Table, _
         sXmlDataNodeSub As String, sXmlFilePath As String, sXmlData As String)

    If sXmlDataNodeSub = "BOResourceWorkplaceOverview1" Then
        oTbl.Sort ExcludeHeader:=True, _
                 FieldNumber:=3, _
                 SortFieldType:=wdSortFieldAlphanumeric, _
                 SortOrder:=wdSortOrderAscending
     End If
End Sub
```

Example: Table filter

```
Sub Main(oDoc As Word.Document, iLoop As Integer, oTbl As Word.Table, _
         sXmlDataNodeSub As String, sXmlFileName As String, sXmlData As String, _
         sXmlDataNode As String, oRow As Word.row)

      If sXmlDataNodeSub = "BOResourceWorkplaceOverview1" Then
        If GetXmlNodeText(sXmlData, sXmlDataNode & "/resource.status[1]") <> "1" Then
            oRow.Delete
        End If
     End If
End Sub
```

Example: Manipulated insertion of HYDRA data references

```
Sub Main(oDoc As Word.Document, iLoop As Integer, oTbl As Word.Table, _
         sXmlDataNodeSub As String, sXmlFilePath As String, sXmlData As String, _
         sXmlDataNode As String, oRow As Word.row, _
         oCc As Word.ContentControl, sCcXmlLink As String, sCcValueBefore As String)

    If (sXmlDataNodeSub = "BOResourceList1") And (sCcValueBefore = "special:yield") Then
        oCc.range.Text = GetXmlNodeText(sXmlData, sXmlDataNode  "/resource.yield[1]") & " " _
                         & GetXmlNodeText(sXmlData, sXmlDataNode & "/resource.unit[1]"
     End If
End Sub
```

3.4.7 Publishing of a completed HYDRA Word Report

Proceed as follows to publish a HYDRA Word Report designed as described above and make it available to other users:

1. Copy your local working copy of the HYDRA Word Report onto the HYDRA server in the `.\`***[HYDRA System]***`\custom\caq\reports` subdirectory. Please create this directory, in case it does not yet exist.
2. Rename the file by removing the underscore at the beginning of the file name. If a file with this file name already exists delete it. Make a backup copy if necessary.
3. Release the corresponding entry in the form management to make the HYDRA Word Report available to other users in the print selection dialog.

4 Documentation of Inspection Results Word Reports

Usage

The certificate shows header data of the inspection requirement as well as inspection results of included characteristics. Data of XML files listed in the section dealing with data sources is used.

Requirements

The certificate is created using the inspectionrequirement_certificate_en.dotm template and the macro library hydramacrolibrary.dotm.

Procedure

Reports are created using the InspectionRequirementExport application that is started by the button “output form” of the initial sample application.

Data sources

1. **root-<Zähler1>_ReqList.xml**
   Includes header data of inspection requirements.
   Zähler1 corresponds to the inspection requirement selected in MOC.
   1.1. **root-<Zähler1>-<Zähler2>_CharList_Req.xml**
      Includes characteristics of the higher-level inspection requirement and characteristic specifications.
      Zähler1 corresponds to the higher-level inspection requirement to which characteristics are assigned
      Zähler2 corresponds to the set of characteristics. In the inspection requirements area this is only one set and, as a result, Zähler2 is always 1.
   1.2. **root-<Zähler1>-1-<Zähler2>_Statistics_Req.xml**
      Includes the statistical values for the corresponding characteristic.
      Subject to the structure of XML files, Zähler2 is always 1.
2. **InspectionRequirement_Certificate_de.xml**
   Includes detail data for print control such as the user name by which the report was requested in MOC.

Structure

The certificate is divided into header area and detail area.

The header area only shows data from **root-<Zähler1>_ReqList.xml**.

The table that is stored there only provides layout functions.

The detail area shows characteristics including corresponding specifications in a structured way within a table.

Root-<Zähler1>-<Zähler2>_CharList_Req.xml is linked to this table as data source.

In the columns “result (xquer)”, “minimum” and “maximum” the cells are merged and include a sub-table listing inspection results. R**oot-<Zähler1>-1-<Zähler2>_Statistics_Req.xml** is linked as data sources to this table.

Characteristic	Unit	Result (values average)	Minimum	Maximum	Result attributive
#LINK:qmcharacteristic.designation	#LINK: unit.id	#LINK: characteristicstatistics.xq	#LINK: characteristicstatistics.xmin	#LINK: characteristicstatistics.xmax	#LINK:qmcharacteristic.inspection_result

Subordinate table

The content of **InspectionRequirement_Certificate_en.xml** is used in the footer only. But it is also available beyond the footer to modify the certificate.

UserExits in use

Only the UserExit **UeFillRowFromXmlAfter** is used. In this UserExit entries of the **CharacteristicList** table are removed from the column “attributive result” (fourth column) if the “variable” value is set in the data element node **/qmcharacteristic.inspection_type.designation_short.**

5 Documentation of Inspection Plan Word Reports

Usage

The report shows header data of an inspection plan as well as the specifications of the included characteristics.

The data of XML files listed in the section dealing with data sources is used.

Requirements

The inspection plan overview is created using the inspectionplan_overview_en.dotm template and the macro library hydramacrolibrary.dotm.

Procedure

Reports are created using the InspectionRequirementExport application that is started by the button “output form” of the initial sample application.

Data sources

XML data sources are structured hierarchically and by counters. There is an XML file with detailed information on this data record for each data record of the correspondingly higher-level XML file.

3. **root-<Zähler1>_InspectionPlanList.xml**
   Includes header data of the inspection plan
   Zähler1 corresponds to the sampling scheme selected in MOC
   1.3. **root-<Zähler1>-<Zähler2>_InspectionPlanCharacteristicList.xml**
   Includes the characteristics from the higher-level inspection plan and their specifications.
   Zähler1 corresponds to the higher-level sampling scheme which the characteristics are assigned to.
   Zähler2 corresponds to the set of characteristics. In the sampling scheme area this is only one set and, as a result, Zähler2 is always 1.

Structure

The report is divided into header area and detail area.
The header area only shows data from **root-<Zähler1>_InspectionPlanList.xml**.
The table that is stored there only provides layout functions.
The detail area shows the characteristics including corresponding specifications in a table.

Root-<Zähler1>-<Zähler2>_InspectionPlanCharacteristicList.xml is linked as data source to this table.

UserExits in use

Only the UserExit **UeFillTableFromXmlAfter** is used. In this UserExit the first column **OP seq No.** of the **Characteristics** table is sorted in ascending order.

6 Documentation of Initial Sample Test Word Report

Usage

The initial sample report shows initial sampling data for the VDA 2.4 report production process and product release. In this context data from the XML files listed in the section dealing with data sources is used.

Requirements

The initial sample report is created using the initialsample_vda24_results_de.dotm template and the macro library hydramacrolibrary.dotm.

Procedure

Reports are created using the InspectionRequirementExport application that is started by the button “output form” of the initial sample application.

Data sources

XML data sources are structured hierarchically and by counters. There is an XML file with detailed information on this data record for each data record of the correspondingly higher-level XML file.

4. **root-<Zähler1>_ReqList.xml**
   Includes header data of the initial sample inspection request.
   Zähler1 corresponds to the initial sample inspection requirement selected in MOC.
   1.4. **root-<Zähler1>-<Zähler2>_CharList_Req.xml**
   Includes the characteristics from the higher-level initial sample inspection requirement and the characteristic specifications.
   Zähler1 corresponds to the higher-level initial sample inspection requirement which the characteristics are assigned to.
   Zähler2 corresponds to the set of characteristics. In the area of initial sample inspection requirements this is only one set and, as a result, Zähler2 is always 1.
   1.5. **root-<Zähler1>-<Zähler2>-<Zähler3>_QMSingleValue_Req.xml**
   Includes the inspection results for the corresponding characteristic. Subject to the structure of XML files, Zähler2 is always 1.

Structure

The initial sample inspection report consists of one cover sheet and x enclosures that include the corresponding inspection results.

The cover sheet shows data from **root-<Zähler1>_ReqList.xml.** There is also an auxiliary tool evaluating data from **root-<Zähler1>-<Zähler2>_CharList_Req.xml**. Further details are described in the section dealing with “UserExits”.
Tables on the cover sheet only provide layout functions and are not linked with differing data sources.

An enclosure sheet is created for every inspection category for which characteristics exist in **root-<Zähler1>-<Zähler2>_CharList_Req.xml**. There is only one page that is copied in the template.
This enclosure template also shows data from **root-<Zähler1>_ReqList.xml** and **root-<Zähler1>-<Zähler2>_CharList_Req.xml**. The **Results** table includes characteristics and is linked with **root-<Zähler1>-<Zähler2>_CharList_Req.xml** as data source.
In the columns “actual values“ “specification met” and “comment” the cells are merged and include a sub-table with inspection results.
Root-<Zähler1>-1-<Zähler2>_QMSingleValue_Req.xml is linked as data source for this table.

Ref No.:	Requirements Specifications	ACTUAL values Supplier	Specification fulfilled yes	 no	Comment
#LINK:inspectionplancharacteristic.op #LINK:inspectionstep.qmcharacteristic.emu_assignment	#LINK: qmcharacteristic.designation #LINK: qmcharacteristic.target_value #LINK: unit.id #LINK:LIMITS_SPECIAL	#LINK:qmsinglevalue.measured_value.value#FORMAT:DECPLACES Subordinate table	☒	☐	#LINK:qmsinglevalue.commentary

Changes to columns of existing tables, table names or adding of new tables have to be taken into account in UserExits with respect to the position IDs that are used there.

Special features

The document is write-protected when macro processing ends so that only content controls can be edited.
Only the enclosures, not the cover sheet are taken into account for page numbering.
The respective page number is reduced by 1 for the cover sheet which, as a result, has page number 0.
This has to be taken into account for printing specific page numbers.

UserExits in use

The following UserExits are used:

1. UeLinkMasterCtrlToXmlBefore

Global variables and functions are defined.

The cover sheet has an auxiliary control element including the special formatting **FORMAT: SPECIALEMU** that is queried in this UE. If this option is set an array including all inspection categories from **root-<Zähler1>-<Zähler2>_CharList_Req.xml** is created, whereas every category is only transferred once, regardless of its occurrence.

The checkboxes for the corresponding enclosures on the cover sheet are set accordingly.

2. UeFillTableFromXmlAfter

In the **Results** table the first column **Ref-Nr.:** is sorted numerically.

3. UeLinkTableCtrlToXmlNotFound

Using a link to non-existing data element nodes **LIMITS_SPECIAL** absolute tolerance limits from **root-<Zähler1>-<Zähler2>_CharList_Req.xml** are converted into relative tolerance limits. Special cases such as identical distances of the upper and lower tolerance to the target value are differentiated in this context. If invalid values are indicated **ERROR** will be displayed instead of the limit value.

Ref No.:	Requirements Specifications	ACTUAL values Supplier	Specification fulfilled yes	 no	Comment
#LINK:inspectionplancharacteristic.op #LINK:inspectionstep.qmcharacteristic.emu_assignment	#LINK: qmcharacteristic.designation #LINK: qmcharacteristic.target_value ToleranceLimits d #LINK:LIMITS_SPECIAL	#LINK:qmsinglevalue.measured_value.value#FORMAT:DECPLACES	☒	☐	#LINK:qmsinglevalue.commentary
		link to non-existing data element node as UserExit entry pooint			

4. UeFillRowFromXmlAfter

At first texts are defined that indicate inspection results that are not valid in **root-<Zähler1>-<Zähler2>_CharList_Req.xml**. Default values are “fail” and “pass”. If other texts are used it has to be adjusted accordingly. Lower-case letters are to be used.

The corresponding checkbox for “specification met“ is now set for the **Values** table.

5. UeOutputReportBefore

For performance reasons, the template is once filled with all inspection results and then copied for every inspection category.

The corresponding checkbox of the inspection category is set and all inspection results that are not assigned to the corresponding inspection category are deleted.

Furthermore, all auxiliary tools included in this UserExit are either deleted or the included text is deleted.

Finally, the focus is set on the first content control that has not been filled out automatically and the document is write-protected. To do so, the password **MPDV** is set.